Parsek Ghost Trajectory Rendering — Design Document

Comprehensive design specification for rendering ghost vessel trajectories during recording playback and re-fly sessions. Covers anchor correction, smoothing, multi-anchor interpolation, DAG propagation across vessel lineages, co-bubble overlap blending, frame-of-reference handling per segment, terrain correction during playback, and edge-case handling.

Parsek is a KSP1 mod for time-rewind mission recording. This document extends the flight recorder, multi-vessel session, and ghost chain systems with the rendering-pipeline algorithm that converts recorded trajectory data into stable, geometrically faithful visual ghosts during playback. It assumes familiarity with the recording DAG, environment and reference-frame taxonomies, ghost chain model, and rewind-to-staging from `parsek-flight-recorder-design.md` *and* `parsek-rewind-to-staging-design.md` *.*

Status: design draft, not yet implemented. Primarily targets the rendering layer (Stages 1-5 in Section 5 are render-time-only). Persistence additions are the minimum needed for cache invalidation, raw-data capture for the surface-terrain and structural-event-snapshot phases, and offset traces for the co-bubble case:

- A new **annotation sidecar** `<id>.pann` (Section 17.3.1) — separate binary file, optional, regenerable. Holds smoothing splines, outlier flags, anchor-candidate UT lists, and co-bubble offset traces. Recordings without it lazy-compute on first use.
- Two `.prec` **schema bumps** (Section 17.3.2) gated by Phase 7 and Phase 9 only — additive per-point fields (`recordedGroundClearance` , `flags`) handled by extending the existing `TrajectorySidecarBinary` version chain. The current chain ends at `BoundarySeamFlagFormatVersion = 8` (already shipped on `main`); Phase 7 reserves v9 (`TerrainGroundClearanceBinaryVersion`) and Phase 9 reserves v10 (`StructuralEventFlagBinaryVersion`).

No `.sfs` save-file shape changes. Recordings of every prior format (v4-v7) remain loadable and playable; pipeline features that depend on absent raw fields degrade gracefully (Section 21.2).

---

1. Introduction

This document specifies the algorithm and edge-case handling for ghost trajectory rendering. The core pipeline (Section 5, Stages 1-5) is render-time-only and does not modify recorded samples. The persistence additions are minimal and additive: a new annotation sidecar (`<id>.pann` , Section 17.3.1) for derived/recomputable data, and two narrow `.prec` schema bumps (Section 17.3.2) for two specific phases that require new raw fields (surface ground clearance, structural-event snapshot flags). No new reference frame is introduced; the existing v5/v6/v7 RELATIVE contracts remain authoritative and the pipeline dispatches through the existing version-specific resolver (Section 7.4). Given the data already captured by the existing recorder, this document defines how ghost trajectories look on screen — and the smallest sidecar surface that supports it.

The motivating problem: recorded absolute trajectory samples have multiple noise sources that produce visible artifacts on playback — high-frequency jitter, alignment errors at separation, and along-track drift across long burns. Naively rendering a ghost at the recorded ABSOLUTE position produces visible bouncing, misaligned attachment to live vessels at separation, and gradual drift through long propulsive segments. Naively rendering relative to a live anchor causes the ghost to follow live player input, breaking the canonical-trajectory invariant.

This document defines a rendering pipeline that resolves all three by combining smoothing, anchor correction, and (where applicable) interpolation between anchors. The pipeline is read-only against `.sfs` save files and game state. It is read-only against existing recorded samples (HR-1) — Phases 7 and 9 add new per-point fields to recordings going forward, but never modify samples that have already been written. Derived data (splines, outlier flags, anchor

candidates, offset traces) lives in a separate annotation sidecar (`<id>.pann`); the canonical `.prec` is touched by exactly two phases (7 and 9) that capture genuinely new raw data.

It covers:

- The three failure modes of naive rendering and the common-mode cancellation principle that addresses them
- The pipeline stages: smoothing, frame transformation, anchor correction, multi-anchor interpolation, co-bubble blend
- Anchor taxonomy: every situation that produces a high-fidelity reference point
- Per-segment frame choice driven by the existing environment taxonomy
- DAG propagation: how anchor corrections flow across vessel lineages
- Co-bubble overlap: live-anchor and ghost-ghost designated-reference cases
- Discontinuities: SOI, TIME_JUMP, bubble entry/exit, environment changes
- Sample-time alignment at structural events
- Continuous terrain correction during surface playback
- Outlier rejection
- Edge cases from gameplay scenarios
- Performance profile
- Implementation map: concept-to-code, pipeline insertion points, sidecar layout — concrete file / type / line refs into the current Parsek codebase
- Implementation phasing: nine ordered phases naming files to create, types to add, hooks to wire, and per-phase done conditions
- Diagnostic logging: per-stage subsystem tags, log lines, levels, and batch counters that make every pipeline decision observable in `KSP.log`
- Test plan: unit tests, log-assertion tests, in-game tests, synthetic recording fixtures, and a per-phase test-done checklist
- Backward compatibility: how recordings of every prior format version (v4-v7) play through the pipeline; lazy-compute path; cross-version re-fly
- Data lifecycle: storage classes for raw samples, annotations, derived caches, and session state
- Post-commit optimization opportunities for pure-ghost playback (no live vessel)
- Hard rules and risk surface across the recorder-to-renderer pipeline

---

2. Design Philosophy

These principles inform every decision in the rendering pipeline.

1. **Recordings are immutable.** The pipeline is a render-time transform. The on-disk recording is never modified by smoothing, anchor correction, or blending. All corrections are applied to a transient render-time copy.

2. **Common-mode cancels in differences.** Two vessels in the same physics bubble share float-precision regime, krakensbane shifts, body-rotation phase, and sample-tick alignment. The difference of their absolute positions is much higher fidelity than either individual position. This is the core mathematical insight the pipeline exploits.

3. **Anchors are universal.** Any moment with a high-fidelity reference is an anchor opportunity: separation events, dock/merge events, RELATIVE-segment entry, orbital-checkpoint boundaries, SOI transitions, bubble entry, designated co-bubble overlap. The "live sibling at re-fly start" case is one instance of a general scheme.

4. **Live reference is a special case of designated reference.** A live (player-controlled) vessel is just one possible primary. Another ghost can serve the same role for ghost-ghost rendering. The pipeline is symmetric over this distinction.
5. **No smoothing across discontinuities.** Every frame change, environment transition, SOI crossing, TIME_JUMP, or bubble entry/exit is a hard segment boundary. Smoothing splines and anchor lerps respect these boundaries.
6. **Per-segment frame choice.** Body-fixed coordinates are natural for surface and atmospheric phases; body-centered inertial is natural for exo-atmospheric phases. The existing environment taxonomy already encodes the right answer.
7. **Vision over precision.** The goal is visual fidelity to the canonical trajectory, not bit-exact reproduction. Sub-meter jitter is the enemy; meter-scale rigid corrections that anchor correctly are acceptable.
8. **Determinism.** Same recordings, same re-fly markers, same UT — same rendered position. The pipeline does not introduce non-deterministic state.

---

3. Failure Modes of Naive Rendering

The pipeline exists to fix three concrete failure modes. Understanding them in isolation is essential before any optimization or change.

3.1 High-Frequency Jitter ("Bouncing")

Cause: physics-tick timing imprecision, single-precision float quantization, krakensbane origin shifts during recording.

Symptom: ghost mesh visibly vibrates frame-to-frame even on smooth coast trajectories. Worse at high orbital velocities — a few milliseconds of timing skew is meters of along-track position.

Scope: any ABSOLUTE-frame trajectory. Worst in EXO_PROPULSIVE at orbital velocities. Negligible in SURFACE_STATIONARY.

3.2 Initial Alignment Error ("Wrong Start Position")

Cause: U_abs and L_abs at the same UT each have several meters of common-mode noise. The noise cancels in their difference but not individually. So even though `U_abs - L_abs` at a separation event is accurate to centimeters, U_abs by itself is meters off from where it should be.

Symptom: at re-fly start, the ghost spawns visibly displaced from the live vessel by 1-10 meters, breaking the "they were attached one moment ago" illusion.

Scope: any case where a ghost should geometrically attach to a live vessel or another ghost at a known event UT.

3.3 Along-Track Frame Drift

Cause: ABSOLUTE recordings stored in body-fixed (rotating) coordinates are sensitive to the planet's rotational phase at the rendered UT. A small UT skew between recording and playback compounds at orbital velocities into meters of displacement.

Symptom: ghost trajectory progressively drifts ahead of or behind its expected path across a long burn or coast. Most visible when there's a downstream anchor — the ghost arrives at the wrong place at the wrong time and snaps into position.

Scope: any ABSOLUTE-frame segment in EXO_*. Driven by total elapsed time and velocity.

3.4 The Naive Relative Trap

If the rendering pipeline tries to compensate for the above by anchoring the ghost to a live vessel's *current* position and applying recorded relative deltas, the ghost moves with player input. Rotating the live vessel rotates the ghost. Translating the live vessel translates the ghost. This violates the canonical-trajectory invariant — the recorded ghost is supposed to be orthogonal to player decisions.

The pipeline must therefore use the relative information without coupling the ghost's motion to the live vessel's live state.

---

4. The Common-Mode Cancellation Principle

For two vessels A and B physically present in the same physics bubble during recording, their absolute samples can be modeled as:

```
A_abs(t) = A_true(t) + noise_common(t) + noise_A(t)
B_abs(t) = B_true(t) + noise_common(t) + noise_B(t)
```

`noise_common` is the shared float-precision frame, krakensbane shift, body-rotation phase, and sample-tick alignment. `noise_A` and `noise_B` are residual per-vessel sampling effects, typically much smaller.

The relative offset:

```
A_abs(t) - B_abs(t) = (A_true(t) - B_true(t)) + (noise_A(t) - noise_B(t))
```

The dominant `noise_common` cancels. What remains is the high-fidelity local relative offset.

Validity condition. Both vessels must be in the same physics bubble at the same UT. As they separate beyond bubble range, they enter different bubble regimes, common-mode breaks down, and the relative offset degrades to the same fidelity as standalone absolute.

This is the basis for all anchor corrections in the pipeline. Wherever the pipeline references "the recorded relative offset at event X," what it's exploiting is common-mode cancellation at that UT.

---

5. Pipeline Overview

The pipeline runs at recording-commit time (smoothing, anchor identification) and at render time (frame transformation, correction lookup, blend). It produces a render-time `U_render(t)` for each ghost from its recorded `U_abs(t)` plus context.

Five stages:

1. **Smoothing** — fit a continuous curve through ABSOLUTE samples per segment. Removes Failure Mode 3.1 (jitter).
2. **Frame transformation** — render-time lift to body-centered inertial for orbital segments, stay body-fixed for surface and atmospheric segments. Removes Failure Mode 3.3 (long-track drift).
3. **Anchor identification** — at every high-fidelity reference point along the trajectory, compute the rigid-translation correction `ε` that aligns the smoothed trajectory with the reference. Addresses Failure Mode 3.2 (alignment error).

4. **Correction interpolation** — between two anchors bracketing a segment, lerp `ε` linearly along the segment. Bounds accumulated drift; further addresses 3.3 across long segments.

5. **Optional co-bubble blend** — within a sub-window where two vessels were physically close in the recording and their stored relative-offset trace exists, render the non-primary as `U_render(t) = P_render(t) + offset(t)`, where `P_render(t)` is the primary's own pipeline output for time `t` (Stages 1+2+3+4 evaluated against the primary's recording — *not* the primary's frozen anchor snapshot, *not* a live vessel's runtime KSP position). The trace gives sub-meter relative geometry; the primary's canonical motion is preserved by going through the pipeline. See Section 10 for the live-primary subtlety that makes this safe against the naive-relative trap.

The render-time output is `U_render(t) = U_smoothed_in_segment_frame(t) + ε(t)`, with the frame transform applied last.

---

6. Stage Details

6.1 Stage 1: Smoothing

Per-segment fit through recorded samples. Catmull-Rom or cubic Hermite splines are appropriate. If the recording stores velocity samples alongside position, prefer Hermite — known tangents produce strictly better fits than tangents estimated from positions alone.

Smoothing acts on the raw recorded samples. It does not cross segment boundaries (Section 11).

The smoother is conservative: it removes sub-meter jitter without flattening real dynamics. The acceleration profile of the smoothed curve must remain consistent with the recorded velocities and the physics regime of the segment — atmospheric burns can have high accelerations, orbital coasts cannot.

Smoothing is a one-time cost per recording, computed at commit time and cached alongside the recording in the sidecar file. Render-time cost is one spline evaluation per ghost per frame.

6.2 Stage 2: Frame Transformation

Per-segment frame choice driven by the existing environment taxonomy:

Segment Environment	Render Frame	Rationale
ATMOSPHERIC	Body-fixed	Frame rotates with the body, atmosphere does too. Lat/lon/alt is the natural representation.
EXO_PROPULSIVE	Body-centered inertial	Orbital-velocity samples in body-fixed are sensitive to rotation timing.
EXO_BALLISTIC	Body-centered inertial (analytical from checkpoints)	Already inertial-frame Kepler propagation.
SURFACE_MOBILE	Body-fixed	Surface coordinate frame is natural.
SURFACE_STATIONARY	Body-fixed	Surface coordinate frame is natural.

For inertial-rendered segments, the pipeline lifts smoothed samples by undoing the planet rotation at the recorded UT, performs all interpolation and correction in inertial coordinates, and re-applies rotation at the current render UT. Small UT skew between recording and playback no longer warps along-track position.

For body-fixed-rendered segments, no lift is needed — samples are stored and rendered in the same frame.

The frame transformation is a render-time operation, performed after spline evaluation and after correction application.

6.3 Stage 3: Anchor Identification

An anchor is a UT at which a high-fidelity reference position for the ghost is available. The pipeline computes one rigid-translation correction `ε` at each anchor.

Anchor types — see Section 7 for the complete taxonomy.

For each anchor at UT t_a:

```
P_target(t_a) := high-fidelity reference position from anchor source
P_smoothed(t_a) := Stage 1 spline evaluation in segment's render frame
ε(t_a) := P_target(t_a) - P_smoothed(t_a)
```

`ε` is a 3-vector translation. It does not include rotation — the canonical trajectory's orientation is preserved.

If multiple anchor sources exist at the same UT (e.g., a dock event involving a live vessel and a checkpoint boundary), they are reconciled by the priority order in Section 7.11.

6.4 Stage 4: Correction Interpolation

A segment may have anchors at one or both endpoints, or none.

Anchor Configuration	Correction Applied
Anchor at start only	`ε(t) = ε_start` (constant)
Anchor at end only	`ε(t) = ε_end` (constant)
Anchors at both ends	`ε(t) = lerp(ε_start, ε_end, (t - t_start) / (t_end - t_start))`
No anchors	`ε(t) = 0`

Interpolation is linear in UT. More sophisticated schemes (e.g., spherical interpolation along the trajectory) are not needed — the correction magnitudes are small and the difference is invisible.

The "no anchors" case is rare in practice. Standalone ghosts with no live reference and no other co-bubble ghost over the entire trajectory would fall into it. They are rendered without correction (Stage 1 smoothing alone), which is the same as today's naive behavior — acceptable since there's nothing to be misaligned with.

6.5 Stage 5: Co-Bubble Overlap Blend

When two vessels were physically close in the original recording (both in the same bubble), the stored relative-offset trace gives sub-meter fidelity for the relative geometry. During playback with one of them as the designated primary (live or ghost), the non-primary is rendered as:

```
U_render(t) = P_render(t) + relative_offset_trace(t)
```

For t in the overlap window. `P_render(t)` is the primary's own pipeline output at time t, computed via Stages 1+2+3+4 against the primary's recording. The primary's canonical motion (orbital advance, recorded burn) is preserved because `P_render` advances with t, not held at an anchor snapshot.

What is "frozen at the anchor UT" is *not* the primary's runtime position. What is frozen is the primary's anchor reference used by Stage 3 to compute the primary's own ε:

- For a **ghost primary**, Stage 3 uses whatever anchor sources Section 7 makes available (orbital checkpoints, RELATIVE boundaries, DAG propagation). `P_render(t)` is the standalone pipeline output.
- For a **live primary** (re-fly), Stage 3 reads the live vessel's spawn-time world position *once* at the anchor UT and uses it as the live-separation anchor (Section 7.1) for the *primary's recording*. After that read, the pipeline never reads the live vessel's runtime position again. `P_render(t)` is computed from the primary's recording exactly as for a ghost primary — see Section 10.4 for why this is safe.

The relative-offset trace is in the same frame as `P_render(t)` (per Section 6.2's per-segment frame choice). Reading live input every frame and substituting it for `P_render(t)` would re-introduce the naive-relative trap (Section 3.4) — which is exactly the failure mode this stage is designed to avoid.

The blend window is bounded by the first of:

- Vessel separation distance exceeds bubble radius or the recorded co-bubble period ends
- A configurable time window after the anchor
- Either vessel destroyed/separated/lost in the recording

A short crossfade at the window boundary blends from the co-bubble formula to the Stage 1+3+4 result. Crossfade is on rendered position only — no data merging.

The co-bubble blend is optional. If it is omitted, the pipeline still produces correct geometry via Stage 1+3+4 — just at smoothing-residual fidelity rather than sub-meter. The blend is the additional refinement that makes very-close-range visuals (debris, plumes, separation effects) look right.

---

7. Anchor Taxonomy

This section enumerates every situation that produces an anchor. Each entry specifies the trigger, the reference position source, and the segment(s) to which the resulting `ε` applies.

7.1 Re-fly Separation Anchor (Live)

Trigger: Player rewinds to a multi-controllable split and re-flies one sibling. The live sibling spawns at the split UT. The ghost siblings begin playback.

Reference position: Live sibling's spawn position plus the recorded relative offset between the two vessels at separation UT. The recorded offset is `ghost_abs(t_sep) - live_abs(t_sep)`, common-mode clean.

Applies to: The ghost segment beginning at the separation event.

7.2 Dock / Merge Anchor

Trigger: A recording contains a dock or merge event at UT t_m, joining two vessels A and B into A+B.

Reference position: The recorded relative offset `A_abs(t_m) - B_abs(t_m)` is common-mode clean. If either A or B has a known position at t_m (live, or already-anchored ghost, or analytical orbit), the other's position is determined by the offset.

Applies to: The segment of the unanchored side ending at t_m, and the segment of the merged result beginning at t_m.

7.3 Undock / EVA / Joint-Break Anchor

Trigger: Same shape as 7.1 but for events occurring within a recording rather than at re-fly start. Two vessels that were previously joined as one separate at UT t_s.

Reference position: The pre-separation single vessel has a known position at t_s (from its segment ending there). Both post-separation siblings inherit position via their recorded body-frame attachment offsets.

Applies to: Both sibling segments beginning at t_s.

7.4 RELATIVE-Segment Boundary Anchor

Trigger: A recording transitions between an ABSOLUTE-frame segment and a RELATIVE-frame segment (typically: approach to a docking range, departure after undock).

Reference position: The RELATIVE-frame segment is anchored to a real persistent vessel and is resolved at render time through the existing version-specific resolver — `TrajectoryMath.ResolveRelativePlaybackPosition` (`Source/Parsek/TrajectoryMath.cs:1049`) — which dispatches by `Recording.RecordingFormatVersion` :

- **v5 and earlier (`< RelativeLocalFrameFormatVersion`):** legacy world-space offset path, `anchorWorldPos + storedOffset` . The pipeline must not auto-reinterpret these as anchor-local data — that is one of the explicit version contract rules (`.claude/CLAUDE.md` → "Rotation / world frame").
- **v6+ (`>= RelativeLocalFrameFormatVersion`):** anchor-local Cartesian offset stored in `TrajectoryPoint.latitude/longitude/altitude` as metres along the anchor's local axes, resolved as `anchorWorldPos + anchorWorldRot * (dx, dy, dz)` via `TrajectoryMath.ApplyRelativeLocalOffset` (`:1007`). The rotation step is mandatory; omitting it misplaces the segment whenever the anchor rotates.
- **v7 (`>= RelativeAbsoluteShadowFormatVersion`):** v6 contract plus the parallel `absoluteFrames` shadow consulted by `ParsekFlight.TryUseAbsoluteShadowForActiveReFlyRelativeSection` (`:15875`) when the live anchor is unreliable during an active Re-Fly session.

The boundary value of the RELATIVE segment, computed via the version-appropriate resolver, is the reference for the adjacent ABSOLUTE segment. New pipeline code never re-implements the resolver formulas inline — it always calls the existing helpers, which remain the single source of truth for the version dispatch.

Applies to: The adjacent ABSOLUTE segment, at the boundary.

7.5 Orbital-Checkpoint Boundary Anchor

Trigger: A recording transitions between an ABSOLUTE-frame propulsive segment and an ORBITAL_CHECKPOINT segment (typically: end of burn into coast, end of coast into burn).

Reference position: Kepler propagation from the checkpoint at the boundary UT. Analytically clean.

Applies to: The adjacent ABSOLUTE segment, at the boundary.

7.6 SOI Transition Anchor

Trigger: A recording crosses a sphere-of-influence boundary.

Reference position: The checkpoint at the SOI transition stores Keplerian elements in the new body's frame. Kepler propagation from this checkpoint provides the reference.

Applies to: Segments adjacent to the SOI checkpoint. SOI transitions are also hard discontinuities for smoothing (Section 11).

7.7 Bubble Entry / Exit Anchor

Trigger: A vessel that was previously outside the recording session's physics bubble enters it (becomes physics-active and starts producing high-fidelity samples) or exits it (becomes propagation-only).

Reference position: At entry, the first physics-active sample is high-fidelity for that vessel because subsequent samples will also be common-mode-correlated with the active focus. At exit, the last physics-active sample is the reference.

Applies to: The propagation-only segment adjacent to the entry or exit.

7.8 Ghost-Ghost Co-Bubble Anchor (Designated Reference)

Trigger: Two recordings R_A and R_B both contain segments where their respective vessels were in each other's physics bubble during the original session (both were physics-active in some ambient session). The stored relative-offset trace exists.

Reference position: One vessel is designated primary (Section 10.1). Its smoothed trajectory plus its anchors determine its rendered position. The other vessel renders relative to the primary using the stored offset trace.

Applies to: Both segments during the co-bubble overlap. Outside the overlap, both fall back to standalone rendering.

7.9 Surface Mobile Anchor (Continuous)

Trigger: A SURFACE_MOBILE segment.

Reference position: The recorded ground clearance plus the current terrain height at the recorded lat/lon. Effectively a continuous anchor at every rendered frame.

Applies to: The SURFACE_MOBILE segment. See Section 13.

7.10 Loop Anchor

Trigger: A looped recording segment is anchored to a real persistent vessel. Already specified in the existing flight recorder doc (looped recordings).

Reference position: The anchor vessel's current position plus the recorded loop-relative offset.

Applies to: The looped segment for its full duration. Loops are continuous-anchor like 7.9.

7.11 Anchor Priority

When multiple anchor types apply at the same UT, priority order:

1. Live reference (7.1)
2. Real persistent-vessel reference (7.4, 7.10)
3. Analytical-orbit reference (7.5, 7.6)
4. DAG-propagated reference from a higher-priority upstream anchor (7.2, 7.3, 9)
5. Designated co-bubble reference (7.8)
6. Bubble entry/exit (7.7)

The highest-priority available source wins. The pipeline does not blend across sources at the same UT.

---

8. Multi-Anchor Interpolation

When a segment has anchors at both endpoints, the correction `ε` is interpolated linearly:

```
ε(t) = ε_start + (ε_end - ε_start) * (t - t_start) / (t_end - t_start)
```

This handles the long-track drift problem. A long EXO_PROPULSIVE segment ending at a dock event would, without end-anchor lerp, accumulate frame drift across the burn and snap into alignment at the dock. With lerp, drift is distributed smoothly across the segment — invisible to the player.

If `ε_start ≈ ε_end`, the interpolation is effectively constant. If they diverge significantly, it indicates either:

- A long segment with significant frame drift — lerp resolves it.
- A genuine recording-vs-playback inconsistency (e.g., physics-tick mismatch over thousands of seconds). Lerp distributes the resulting error across the segment, which is the best the pipeline can do without modifying the canonical trajectory.

The interpolated correction does not introduce visible motion artifacts — it's a slow, smooth, sub-meter-per-second shift that's invisible against the trajectory's actual motion.

---

9. DAG Propagation

Vessel lineages branch and merge across the recording DAG. A single live anchor at a re-fly point determines corrections for arbitrarily complex downstream ghost lineages.

9.1 Propagation Rule

When a segment ends at a structural event (split or merge) with another segment beginning there, the recorded relative offset between them at that event is common-mode clean. The end-anchor `ε` of the ending segment determines the start-anchor `ε'` of the beginning segment via:

```
ε' = ε + (recorded_offset_at_event - smoothed_offset_at_event)
```

Where `smoothed_offset_at_event` is the difference of the two segments' smoothed positions at the event UT. The second term corrects for any residual smoothing-induced offset at the event.

9.2 Example: Three-Stage Rocket Re-Fly

L1 → L2 → U with L1 re-flown live. Original session had U as active focus with L1 and L2 background-recorded.

Anchor chain:

- L1 is live. No correction needed (it's the reference).
- At t_1 (L1/L2+U separation): L1's position is live ground truth. L2+U's start-anchor is computed from `L1_live + recorded_offset(t_1)`. This produces `ε_{L2+U,start}`.
- L2+U is a single vessel until t_2. Its segment may have any other downstream anchor (e.g., engine start). If so, lerp; if not, `ε(t) = ε_{L2+U,start}`.
- At t_2 (L2/U separation): L2+U's end-anchor at t_2 is `ε(t_2)` from above. L2 and U each get their start-anchor at t_2 via the recorded offsets between L2+U (single ending vessel) and L2, U (two beginning vessels).
- L2 segment may end at crash or escape; U segment continues into orbit.

A single live anchor on L1 determines corrections for the entire downstream lineage — L2+U, L2, and U — via DAG edges.

9.3 Cross-Recording Propagation

When a recording R_A's chain tip docks into a vessel claimed by recording R_B, R_A's anchored position at the dock event determines a start-anchor for R_B's continuation segment. Same propagation rule.

This makes the multi-recording, multi-tree case work without special handling — every dock/merge event in the ghost chain walker is also an anchor opportunity.

9.4 Suppressed Subtree Handling

Ghost playback already filters by session-suppressed subtree (existing rewind-to-staging mechanism). Suppressed segments are not rendered. They do not propagate anchor corrections — a suppressed segment's anchors are not consulted by downstream non-suppressed segments. If a non-suppressed segment had a suppressed predecessor, its start-anchor type falls back to whatever non-suppressed reference is available (live, designated co-bubble, or no anchor).

9.5 Cycles and Termination

The DAG is acyclic by construction. Anchor propagation walks edges in trajectory time-order from each anchor source outward. Each segment's `ε` is computed at most twice — once at start, once at end. No cycle protection is needed.

Propagation terminates at:

- Segments with no downstream successors (e.g., crashed vessels).
- Segments whose successor is suppressed (Section 9.4).
- Recording ends (terminal state).

---

10. Co-Bubble Overlap

10.1 Designated Primary

When two co-bubble vessels are both ghosts (no live reference), the pipeline designates one as primary. Selection rule, in priority order:

1. The vessel with a live anchor in its DAG ancestry (closest to the live re-fly point).
2. The vessel committed earlier in the timeline.
3. The vessel with a higher sample rate during the overlap.
4. Tiebreaker: stable order by recording ID.

The primary's position is computed by Stage 1+2+3+4 standalone. The non-primary is rendered relative to the primary using the stored offset trace.

When a live vessel is in the bubble, it is always primary by Section 7.11.

10.2 Stored Offset Trace

The relative-offset trace is computed from the recorded absolute-difference vector `Δ_world(t) = non_primary_abs(t) - primary_abs(t)` and persisted in a primary-aligned frame: at each sample UT, transform `Δ_world(t)` into the primary's body-centered inertial frame at that UT (`Δ_primary_inertial(t) = Inverse(primary_body_rotation(t)) * Δ_world(t)`). Persisting in the primary's inertial frame removes body-rotation phase from the trace; at playback the inertial offset is re-lowered into world coordinates against the playback UT's body rotation, then added to `P_render(t)` . For `Atmospheric` / `Surface*` primary segments where the rendered frame is body-fixed (Section 6.2), the trace is persisted in the body-fixed frame instead — no inertial round-trip — because the body rotation phase cancels naturally between recording and playback in the body-fixed frame.

The trace is bounded by:

- Both vessels were in the same physics bubble in the original session.
- Sample rate is the lower of the two (resampled).
- The trace's frame is pinned per-segment by the primary's render frame (Section 6.2). Trace samples carry a one-byte frame tag (`inertial` / `body-fixed`) so the playback-time un-lift is unambiguous.

If either vessel was the active focus in the original session, the offset has the lowest possible noise — only the non-active vessel's sampling residual contributes; the active one's is essentially zero relative to its own frame.

10.3 Trace Validity Boundaries

The trace becomes invalid at:

- Either vessel destroyed or structurally split in the recording.
- Bubble exit (separation distance grows beyond bubble radius).
- TIME_JUMP event on either vessel.

At a validity boundary, the rendering reverts to standalone (Stage 1+3+4). A short crossfade smooths the visual transition.

10.4 Live Co-Bubble Case

When the primary is a live vessel, the formula is `U_render(t) = P_render(t) + offset(t)` exactly as for ghost-ghost. The subtlety is what `P_render(t)` evaluates to.

The live vessel was being recorded right up to the re-fly start UT — its recording exists. After re-fly start the player is flying it live, but for pipeline purposes the recording is what defines the canonical trajectory. `P_render(t)` is computed by running the live vessel's recording through Stages 1+2+3+4, the same way a ghost primary's `P_render(t)` is computed.

The live read happens once: at the anchor UT, Stage 3 reads the live vessel's world position and stores it as the live-separation anchor (Section 7.1) for the primary's own recording. From that point on, the pipeline never reads the live vessel's runtime KSP state again. `P_render(t)` is a function of the recording, the smoothing splines, and the anchor map — none of which contain live state past the anchor UT.

The result: peer ghost `U_render(t)` follows the recorded canonical path of `P` (orbital advance, recorded burn) plus the recorded relative offset. Player input on the live vessel — rotating the camera, throttling differently than recorded, decoupling at a different UT — does not move the peer ghost, because the pipeline's view of `P` is the recording, not the live vessel.

If the player diverges from the recording (deliberate re-fly choice), the live vessel and its `P_render(t)` will visibly separate. That is the correct visual: the ghost remains anchored to the canonical recording while the live vessel demonstrates the new trajectory.

---

11. Discontinuities

Hard discontinuities are points across which smoothing splines and correction interpolation must not cross. Each discontinuity is a segment boundary in the smoother and a fresh anchor opportunity.

Discontinuity Type	Source	Anchor on Both Sides?
Structural event (split, merge, dock, undock)	DAG	Yes
Environment transition (e.g., ATMOSPHERIC -> EXO_PROPULSIVE)	Environment taxonomy	Where reference exists

Reference-frame transition (e.g., ABSOLUTE <-> RELATIVE)	Reference-frame taxonomy	Yes (RELATIVE side is exact)
SOI transition	Body change	Yes (SOI checkpoint)
Bubble entry / exit	Physics-active state change	Yes (last/first physics sample)
TIME_JUMP	Relative-state time jump	Each side has stored state vector
Recording start / end	Recording boundary	Start: depends on context. End: always (terminal state).

Smoothing splines fit each region between discontinuities independently. Correction interpolation respects the same boundaries — no lerp crosses a discontinuity, even if anchors exist on both sides.

A hard discontinuity is generally invisible to the player — switching between body-fixed and inertial rendering produces no visible change because the world is in the same place; a TIME_JUMP is visible as a deliberate cut, not an artifact.

---

12. Sample-Time Alignment at Structural Events

For common-mode cancellation to produce a clean offset at a structural event, the two vessels' samples at the event UT must be synchronously taken.

In the recorder:

- Active recording samples at physics-tick rate.
- Background recording samples at proximity-based rate.
- Physics ticks for the active vessel and proximity-tick alignment for background vessels are not guaranteed to coincide with structural-event UTs.

Without alignment, the recorded `A_abs(t_event)` and `B_abs(t_event)` are interpolated from samples on either side, and the interpolation's noise is not common-mode.

Rule: at every structural event (split, merge, dock, undock, joint-break), the recorder produces a mandatory snapshot at the exact event UT for every involved vessel. If a tick boundary doesn't align, sub-tick interpolation is performed before recording — once, at recording time, where the interpolation can use the same physics state for both vessels.

This produces a single common-mode-correlated pair of samples at every structural-event UT. The anchor at that event is then computable to physics-precision.

This is a recorder requirement, not a renderer requirement, but it's listed here because the rendering pipeline depends on it. If recordings predate this rule, anchor corrections at events may have a few-tick worth of additional noise — degraded but not broken.

---

13. Surface and Terrain

13.1 Continuous Terrain Correction

For SURFACE_MOBILE segments, KSP's procedural terrain can shift sub-meter between sessions. The static spawn-time terrain correction (already specified in the flight recorder doc) is sufficient for endpoints. For continuous playback (a rover driving across terrain), every rendered frame needs terrain-aware altitude.

Rule: every SURFACE_MOBILE trajectory sample carries a recorded ground-clearance value alongside altitude. At render time:

```
rendered_altitude = current_terrain_height(lat, lon) + recorded_ground_clearance
```

Lat and lon come from the smoothed body-fixed trajectory. Terrain raycast is performed at render time, ideally cached by lat/lon bucket to amortize across frames.

13.2 Terrain Raycast Cost

Raycasting per ghost per frame is acceptable for a small number of surface ghosts. For dozens of co-located surface ghosts (a base with rover traffic), raycasting would dominate. Cache strategies:

- Per-frame cache keyed by lat/lon at meter granularity.
- Per-segment pre-pass that raycasts at every recorded sample position once, stores the delta between current and recorded terrain, and applies the delta as a continuous correction along the smoothed trajectory.

The pre-pass approach makes terrain correction structurally identical to other anchor corrections — just with anchors densely placed along the segment.

13.3 Terrain Failure Modes

If the raycast misses (hole in collider mesh, transient loading), fall back to recorded altitude. A miss is rare and produces at most a frame of altitude pop, which is preferable to the alternative of clipping into terrain.

---

14. Outlier Rejection

14.1 Pre-Smoothing Outlier Filter

Recorded samples may contain physics-glitch outliers (kraken events, transient single-frame teleports). Smoothing splines fitted through outliers produce visually catastrophic curves.

Rule: before fitting the smoothing spline, scan samples for physical implausibility using thresholds appropriate to the segment's environment:

- Implied acceleration substantially above the physical regime's plausible maximum (atmospheric chemistry-rocket maxima for ATMOSPHERIC, lower for EXO_*).
- Single-tick position delta exceeding bubble radius.
- Altitude outside the body's plausible range (slightly below sea level to body SOI radius).

Samples violating any threshold are rejected. The smoothing spline is fitted on the cleaned set.

Specific threshold values are tuning parameters and are deferred (Section 22.1).

14.2 Outlier Persistence

Rejection is render-time only. The on-disk recording is not modified — the outliers remain in the canonical record (they are part of what really happened in the original session). Replays of the same recording always reject the same outliers, deterministically.

14.3 Multi-Outlier Clusters

If a substantial fraction of a segment's samples are rejected, the segment is flagged as low-fidelity in diagnostics. The pipeline still renders it but logs a warning. Sustained outlier clustering indicates a recording quality issue worth

investigating.

---

15. Edge Cases from Gameplay Scenarios

This section walks through scenarios from gameplay and maps each to the pipeline's handling. Each scenario is intentionally specific and tests a different combination of pipeline features.

15.1 Two Ghosts in Formation, No Live Vessel

Two committed recordings with vessels in a tight formation. Player rewinds before both, no re-fly.

Handling: ghost-ghost co-bubble (Section 10). Designated-primary selection per Section 10.1. The primary renders standalone (Stages 1+2+3+4 with whatever anchors apply); the non-primary renders relative via the stored offset trace. Formation geometry is preserved to centimeter fidelity.

15.2 Orbital Rendezvous Replay

R1 launches station S; R2 launches approach vessel A that coasts to S, switches to RELATIVE-frame within docking range, docks. Player rewinds during R2.

Handling:

- A's launch and ascent: ABSOLUTE, smoothed, anchored at separation from launch vehicle (live or ghost).
- A's coast to S: ABSOLUTE, smoothed, end-anchored at the RELATIVE-segment boundary.
- A's RELATIVE-frame approach: rendered exactly as `S_position + recorded_offset` per existing RELATIVE rules.
- A's dock event: anchor that propagates to S+A's continuation if any.

The lerp of the correction across A's coast eliminates the previous "snap into RELATIVE alignment" artifact.

15.3 Multi-Stage Rocket Re-Fly

L1 → L2 → U. Re-fly L1 live. See Section 9.2.

15.4 Surface Vehicles Near a Base

R1 launches base B. R2 drives a rover near B. Player re-flies a different rover near B; R2 plays as ghost.

Handling: R2's trajectory is SURFACE_MOBILE, body-fixed, with continuous terrain correction (Section 13). If R2 was ever in B's physics bubble during R2's recording, ghost-ghost co-bubble applies; otherwise standalone.

15.5 Long Coast With Boundary Anchors

EXO_PROPULSIVE burn → EXO_BALLISTIC coast (ORBITAL_CHECKPOINT) → EXO_PROPULSIVE burn → EXO_BALLISTIC coast.

Handling: each ORBITAL_CHECKPOINT segment is analytically clean — Kepler propagation, no smoothing. Each EXO_PROPULSIVE segment is bracketed by checkpoint anchors at both ends. Stage 4 lerp interpolates the correction across the burn. No accumulated drift visible.

15.6 SOI Transition Mid-Burn

EXO_PROPULSIVE burn straddling Kerbin/Mun SOI.

Handling: SOI checkpoint is a hard discontinuity. Pre-SOI samples are smoothed and anchored within Kerbin's inertial frame. Post-SOI samples are smoothed and anchored within Mun's inertial frame. The SOI checkpoint provides the

anchor for both sides.

15.7 Brief Out-of-Bubble Excursion

Sibling drifts slightly beyond bubble radius and back during the original recording.

Handling: bubble exit is a hard discontinuity. The out-of-bubble period uses orbital propagation (existing checkpoint mechanism). Bubble re-entry is a fresh anchor opportunity (the resumed sample is high-fidelity).

15.8 Sample Alignment at Structural Events

See Section 12.

15.9 Very Brief Overlap

Sibling disintegrates a fraction of a second after separation due to aero stress.

Handling: co-bubble blend window terminates at vessel destruction (Section 10.3). Stage 3 anchor correction at separation is still valid for the few samples that exist. Falls back gracefully to standalone smoothing for any remaining trajectory data.

15.10 Kraken Outliers

Physics glitch teleports a vessel to an absurd distance for one frame.

Handling: outlier rejection (Section 14) drops the bad sample before smoothing. The spline fits through neighboring valid samples without distortion.

15.11 Time Warp Transitions

Player time-warps through a ghost playback.

Handling: TIME_JUMP events are hard discontinuities. Rendering at warp rate uses analytical propagation where available (ORBITAL_CHECKPOINT), or skipped frames where it isn't (high-rate ABSOLUTE samples are not interpolated to warp UT — the ghost simply doesn't render between sample times during warp). The per-frame anchor correction lookup is unchanged; the lerp interpolation evaluated at the current warp UT yields the right value.

15.12 Save/Load Mid-Playback

Player saves and loads while a ghost is mid-trajectory.

Handling: pipeline state is not persisted. Rebuild from recording data on every load. Anchor corrections are deterministic functions of (recordings, current re-fly markers, current UT) — same inputs, same outputs.

15.13 Many Ghosts in Bubble

Many ghosts simultaneously loaded in physics bubble (a base with traffic, a debris field, a constellation pass).

Handling: Stage 1 smoothing is precomputed at commit. Stage 2 frame transformation is a single matrix multiply per ghost. Stage 3 anchors are looked up. Stage 4 lerp is a single multiply. Stage 5 co-bubble blend applies only to ghost pairs designated as primary/non-primary; each ghost is non-primary in at most one pair.

Per-frame cost per ghost: one spline eval, one matrix multiply, one anchor lookup, one lerp, optionally one offset-trace lookup. All small and bounded.

Existing ghost soft caps and zone-based culling apply unchanged.

15.14 Looped Ghost Near Live Vessel

A looped recording (e.g., rover doing laps around a base) plays back while the player flies a real vessel near the base.

Handling: looped segments are continuous-anchor (Section 7.10). The loop's anchor is the persistent vessel; the looped trajectory renders exactly as `anchor_vessel_position + loop_offset(loop_phase)`. No smoothing or correction needed — the loop anchor is exact at every frame. If the loop's vessel is also co-bubble with another ghost, designated-primary rules apply.

15.15 Ghost in Atmospheric Re-entry

A ghost re-enters atmosphere during playback while no live vessel is nearby.

Handling: ATMOSPHERIC segment, body-fixed frame, smoothed. Heating/drag effects in the original recording are baked into the recorded trajectory shape — the ghost faithfully replays them. Plume effects, glow, etc. are existing rendering concerns separate from the trajectory pipeline.

15.16 Mid-Mission Save File Sharing

A player loads another player's saved game with committed Parsek recordings.

Handling: anchor corrections are recomputed from scratch on load (Section 16.2). The shared save's recordings produce the same rendered ghosts on both players' machines, modulo terrain raycast results (Section 13). Visual consistency between players is therefore best-effort, not guaranteed at sub-meter precision.

15.17 Recording Predates Sample-Time Alignment

A player loads a save with recordings made before the recorder started producing structural-event-aligned snapshots (Section 12).

Handling: anchor corrections at structural events use whatever samples exist nearest the event UT. Common-mode cancellation is degraded by the inter-tick interval but still produces meter-scale rather than naive-tens-of-meters errors. No special handling — the pipeline degrades gracefully.

---

16. Performance

16.1 Cost Profile

Stage	When	Cost
Smoothing	Recording commit	Linear in sample count, one-time per segment
Anchor identification	Recording commit + re-fly start	One vector subtraction per anchor
Frame transformation	Render time	One matrix multiply per ghost per frame
Spline evaluation	Render time	Constant per ghost per frame
Lerp	Render time	One multiply-add per ghost per frame
Co-bubble offset lookup	Render time	One spline eval per overlapping pair per frame

Terrain raycast	Render time (surface)	One physics raycast per surface ghost per frame, cacheable

The total per-ghost per-frame budget is small and bounded. Existing zone-based culling (Zone 3+ ghosts not rendered) and soft caps continue to dominate large-scene budgets.

16.2 Cached Data

Per recording (sidecar):

- Smoothing spline coefficients per segment.
- Outlier-rejection result per segment (which samples were rejected).
- Co-bubble offset traces for any vessel pair with overlap (lazy or eager).

Per re-fly session (transient, in-memory):

- Anchor `ε` values per ghost per anchor.
- Designated-primary assignments for active co-bubble pairs.
- Frozen reference snapshots for live anchors.

Rebuilt on session entry. Not persisted.

16.3 Cache Invalidation

Cached splines and outlier results invalidate when:

- The recording's underlying samples change (commit time only — recordings are immutable post-commit, so this is once).
- The smoothing or outlier algorithm version bumps (would be a sidecar version-header change).

Anchor `ε` values invalidate when:

- Re-fly session starts, ends, or transitions.
- A live anchor's reference vessel respawns (terrain correction can shift the spawn slightly).
- A new committed recording adds a downstream anchor.

Designated-primary assignments invalidate when:

- A new ghost enters or exits the bubble.
- A primary ghost terminates.

---

17. Implementation Map

This pipeline is a rendering-layer addition. It does not modify any existing data structures or persisted formats beyond additive sidecar fields. The following tables anchor each abstract concept to a concrete type, file, or line range in the current Parsek codebase. New code is added to the named seams; existing types are extended only via additive fields.

17.1 Concept-to-Code Map

Concept (this doc)	Concrete artifact	File / line
Trajectory sample	`struct TrajectoryPoint`	`Source/Parsek/TrajectoryPoint.cs:`

Segment	`struct TrackSection`	`Source/Parsek/TrackSection.cs:50`
Environment taxonomy	`enum SegmentEnvironment`	`Source/Parsek/TrackSection.cs:21`
Reference-frame taxonomy	`enum ReferenceFrame`	`Source/Parsek/TrackSection.cs:34`
Section provenance	`enum TrackSectionSource`	`Source/Parsek/TrackSection.cs:10`
Structural event (split/merge/dock/etc.)	`class BranchPoint`	`Source/Parsek/BranchPoint.cs:18`
Mid-segment events (controller, crew, time-jump)	`struct SegmentEvent`	`Source/Parsek/SegmentEvent.cs:18`
Orbital checkpoint	`struct OrbitSegment`	`Source/Parsek/OrbitSegment.cs:9`
Recording	`class Recording` implementing `IPlaybackTrajectory`	`Source/Parsek/Recording.cs`
Trajectory boundary	`interface IPlaybackTrajectory`	`Source/Parsek/IPlaybackTrajectory`
Positioning boundary	`interface IGhostPositioner`	`Source/Parsek/IGhostPositioner.cs`
Engine	`GhostPlaybackEngine`	`Source/Parsek/GhostPlaybackEngine`
Flight-scene positioner	`ParsekFlight : IGhostPositioner`	`Source/Parsek/ParsekFlight.cs`
Pure trajectory math	`static class TrajectoryMath`	`Source/Parsek/TrajectoryMath.cs:2`
Re-fly session marker (live anchor)	`class ReFlySessionMarker`	`Source/Parsek/ReFlySessionMarker.`
Existing absolute-shadow fallback	`TryUseAbsoluteShadowForActiveReFlyRelativeSection` / `ResolveAbsoluteShadowPlaybackFrames`	`Source/Parsek/ParsekFlight.cs:158`
RELATIVE-frame world resolver	`TryResolveRelativeWorldPosition` / `TryResolveRelativeOffsetWorldPosition`	`Source/Parsek/ParsekFlight.cs:156`
Map / tracking-station ghosts	`class GhostMapPresence`	`Source/Parsek/GhostMapPresence.cs`
Camera follow	`class WatchModeController`	`Source/Parsek/WatchModeController`
Chain segment state	`class ChainSegmentManager`	`Source/Parsek/ChainSegmentManager`

Recorder	`class FlightRecorder`	`Source/Parsek/FlightRecorder.cs`
Ballistic tail extension	`BallisticExtrapolator` + `IncompleteBallisticSceneExitFinalizer`	`Source/Parsek/BallisticExtrapolat` `IncompleteBallisticSceneExitFinal`
Patched-conic snapshot	`PatchedConicSnapshot`	`Source/Parsek/PatchedConicSnapsho`
Sidecar I/O	`TrajectorySidecarBinary` (binary `.prec` codec) + `RecordingSidecarStore` (load / commit orchestration)	`Source/Parsek/RecordingStore.cs:5` (format constants), `Source/Parsek/TrajectorySidecarBi` (`Write`), `:184` (`Read`), `:646-649` (`absol` write), `:677` (`absoluteFrames` read), `Source/Parsek/RecordingSidecarSto` `235` (load probe / read), `:729` (com staging)
Path validation	`RecordingPaths.ValidateRecordingId`	`Source/Parsek/RecordingPaths.cs`
Safe-write file I/O	`FileIOUtils`	`Source/Parsek/FileIOUtils.cs`
Save / load wiring	`ParsekScenario OnSave/OnLoad`	`Source/Parsek/ParsekScenario.cs`
Logging	`ParsekLog.Info` / `Warn` / `Verbose` / `VerboseRateLimited` / `VerboseOnChange` / `TestSinkForTesting`	`Source/Parsek/ParsekLog.cs`
Test generators	`RecordingBuilder`, `VesselSnapshotBuilder`, `ScenarioWriter`	`Source/Parsek.Tests/Generators/`

17.2 Pipeline Insertion Points

Per-stage hook locations in the existing playback path. Each hook is additive; existing call sites remain unchanged.

Stage	Where it runs	Existing seam to hook
1. Smoothing (commit-time, cached)	`RecordingStore` finalize path during commit	New private method, e.g. `FitSmoothingSplinesPerSection(Recording rec)`. Called once per recording at commit. Result stored in a new `TrackSectionAnnotations` sidecar (Section 17.3).
1. Smoothing (lazy, missing annotation)	`RecordingStore` load path, after `DeserializeTrackSections`	Same routine triggered when annotation node is absent.
2. Frame transformation	Render-time, inside positioner	New `TrajectoryMath.LiftToInertial` / `LowerFromInertial` helpers; called by `ParsekFlight.InterpolateAndPosition` for `EXO_PROPULSIVE` / `EXO_BALLISTIC` sections.
3. Anchor identification	Commit-time pass over `TrackSections` +	New `AnchorCandidateBuilder` populating `TrackSectionAnnotations.AnchorCandidateUTs`.

(static)	`BranchPoints`	
3. Anchor identification (dynamic, live)	Re-fly start (`ReFlySessionMarker` written) and on RP load	Hook in `ParsekScenario.OnLoad` and `RewindInvoker.ConsumePostLoad`. Build `RenderSessionState.AnchorCorrections` map keyed by `(recordingId, sectionIndex, side)`.
4. Correction interpolation	Render-time, per ghost	Wraps the spline evaluation inside `ParsekFlight.InterpolateAndPosition`. Reads `RenderSessionState.AnchorCorrections`; lerps by UT.
5. Co-bubble blend	Render-time, per pair	New `CoBubbleBlender` invoked from `ParsekFlight.InterpolateAndPosition` when both ghosts of a designated pair are loaded; otherwise no-op.
Outlier rejection (Section 14)	Smoothing prelude	Same routine as Stage 1; produces `OutlierFlags` annotation.
Surface terrain correction (Section 13)	Render-time, after Stage 1	Existing `ParsekFlight.PositionAtPoint` / `PositionAtSurface` already consult terrain via the standard `body.GetWorldSurfacePosition/PQS` path; pipeline adds a `recordedGroundClearance` field to `TrajectoryPoint` (additive, defaults to NaN for legacy points).

17.3 Sidecar Layout

The pipeline persists data in two distinct ways. The split is deliberate: `.prec` is a high-frequency-loaded binary file with its own version-stamped schema (`Source/Parsek/TrajectorySidecarBinary.cs` , magic `PRKB` , current version `RelativeAbsoluteShadowFormatVersion = 7`); pipeline annotations are derivable from raw recordings and live in a separate binary sidecar so the canonical `.prec` is touched only when actually new *raw* fields are needed.

17.3.1 New Annotation Sidecar `<id>.pann`

A new binary sidecar parallels `<id>.prec` , written via a new `PannotationsSidecarBinary` static class (mirrors `TrajectorySidecarBinary` 's shape). Its existence is *optional* — a recording without `.pann` is loaded normally and the pipeline computes annotations on first use. The file lives under `saves/<save>/Parsek/Recordings/<id>.pann` ; pathing routes through a new `RecordingPaths.BuildAnnotationsRelativePath` helper that reuses the existing `ValidateRecordingId` guard.

Binary schema (initial version `PannotationsBinaryVersion = 1`):

```
[0..3]    Magic "PANN" (4 bytes ASCII)
[4..7]    PannotationsBinaryVersion (int32)
[8..11]   AlgorithmStampVersion (int32) — bumps when smoothing / outlier classifier algorithms change
[12..15]  SourceSidecarEpoch (int32) — matches the source .prec's SidecarEpoch (cache-key for self)
[16..19]  SourceRecordingFormatVersion (int32) — matches the source .prec's CurrentRecordingFormatVersion at write time
[20..51]  ConfigurationHash (32 bytes) — SHA-256 over the canonical-encoded tunable-parameters block; see "Configuration Cache Key" below
```

```
[52..]    RecordingId (length-prefixed UTF-8 string)
[..]      String table (count + strings) — mirrors .prec's table layout
[..]      SmoothingSplineList
            count : int32
            entries[count] :
              sectionIndex : int32
              splineType   : byte           (0=CatmullRom, 1=Hermite — extensible)
              tension      : float32
              knotCount    : int32
              knots        : double[knotCount]
              controlsX    : float32[knotCount]
              controlsY    : float32[knotCount]
              controlsZ    : float32[knotCount]
              frameTag     : byte           (0=body-fixed, 1=inertial — Section 6.2)
[..]      OutlierFlagsList
            count : int32
            entries[count] :
              sectionIndex   : int32
              classifierMask : byte
              packedBitmap   : length-prefixed byte[] (one bit per sample)
              rejectedCount  : int32
[..]      AnchorCandidatesList
            count : int32
            entries[count] :
              sectionIndex : int32
              utCount      : int32
              uts          : double[utCount]
              types        : byte[utCount]   (matches AnchorSource enum, Section 18 Phase 2)
[..]      CoBubbleOffsetTraces
            count : int32
            entries[count] :
              peerRecordingId            : length-prefixed UTF-8 string
              peerSourceFormatVersion    : int32   — peer .prec's
CurrentRecordingFormatVersion at trace-compute time
              peerSidecarEpoch           : int32   — peer .prec's SidecarEpoch at trace-
compute time
              peerContentSignature       : 32 bytes — SHA-256 over the peer's raw sample bytes
covering [startUT, endUT]
              startUT, endUT             : double, double
              frameTag                   : byte    — 0=body-fixed, 1=inertial (Section 10.2)
              sampleCount                : int32
              uts                        : double[sampleCount]
              dx, dy, dz                 : float32[sampleCount] (each axis as parallel array)
              primaryDesignation         : byte    — 0=self, 1=peer (Section 10.1)
```

Configuration Cache Key. The `ConfigurationHash` field is SHA-256 over a canonical-encoded record of every tunable that affects derived output. The encoding is endian-fixed and field-order-fixed so the hash is reproducible across machines and runs. Tunables included:

Tunable	Owner	Affects

`smoothingSplineType` (enum byte)	`Rendering/SmoothingSpline.cs`	`SmoothingSplineList`
`smoothingTension` (float32)	same	`SmoothingSplineList`
`outlierClassifierThresholds` (float32 vector + bounds)	`Rendering/OutlierClassifier.cs`	`OutlierFlagsList`; canonical bytes `[21..24]` atmospheric accel, `[25..28]` exo-propulsive accel, `[53..56]` exo-ballistic accel, `[57..60]` SurfaceMobile accel, `[61..64]` SurfaceStationary accel, `[65..68]` Approach accel, `[69..72]` bubble-radius cap, `[73..76]` altitude floor, `[77..80]` altitude ceiling margin, `[81..84]` cluster-rate threshold.
`anchorPriorityVector` (byte[10])	Section 7.11	`AnchorCandidatesList` ordering
`coBubbleBlendMaxWindow` (float64 s)	`Rendering/CoBubbleBlender.cs`	`CoBubbleOffsetTraces` window
`coBubbleResampleHz` (float32)	same	trace UT density
`useAnchorTaxonomy` (bool, byte)	`ParsekSettings.cs`	`AnchorCandidatesList` (off → empty block; on → populated). Phase 6 follow-up — without this byte, flipping the rollout flag would let a previously-cached `.pann` with an empty candidate list cache-hit a flag-on session, breaking HR-10 freshness.
`useOutlierRejection` (bool, byte)	`ParsekSettings.cs`	`OutlierFlagsList` (off -> empty block; on -> populated), stored at canonical byte `[85]` so flipping the rollout flag invalidates cached `.pann` files.

Any code path that reads a tunable must contribute it to the canonical encoding. Missing one is the bug HR-10 names: a parameter change with no cache invalidation. Tests verify reproducibility (same inputs → same hash) and sensitivity (perturbing any tunable → hash changes) — see Section 20.1's `PannotationsConfigHashTests`.

Per-Trace Peer Validation. A `CoBubbleOffsetTrace` is derived from two raw sample series: the source recording's and the peer's. The header `SourceSidecarEpoch` covers freshness against the source; per-trace fields cover freshness against the peer. At consumption time:

1. Resolve the peer recording by `peerRecordingId`.
2. If the peer is missing or its `.prec` `CurrentRecordingFormatVersion` differs from `peerSourceFormatVersion`, discard the trace and recompute (peer was healed / migrated / re-recorded).
3. If the peer's `.prec` `SidecarEpoch` differs from `peerSidecarEpoch`, discard and recompute (peer's sidecar was rewritten — supersede commit, repair, etc.).
4. Optionally (defence-in-depth, recommended), recompute `peerContentSignature` over the peer's raw samples covering `[startUT, endUT]` and compare. Mismatch → discard and recompute. The signature catches edits that bypassed the epoch counter.
5. Only when all four pass is the trace accepted; otherwise the consumer falls back to standalone rendering (HR-9 visible-failure path) and emits a `Pipeline-CoBubble` Info log line naming the recompute reason.

Without this per-trace key, a peer whose `.prec` was rewritten — by `SupersedeCommit`, by a healing pass, by a re-recording, or by any future format migration — would bind to stale offsets and render geometry off the wrong canonical path. The per-trace cache key is the only thing that closes that gap (HR-10, HR-14).

Properties:

- The file is regenerable. A reader encountering a non-matching `AlgorithmStampVersion`, `ConfigurationHash`, or `SourceSidecarEpoch` discards the entire `.pann` and triggers lazy recompute (HR-10). Per-trace mismatches in the co-bubble block discard only the affected trace, not the whole file — splines and outlier flags depend only on the source recording and remain valid.
- Both probe and load go through a `PannotationsSidecarProbe` struct in the same shape as `TrajectorySidecarProbe`. Missing file surfaces as `Success = false, FailureReason = "file missing"` → expected, not an error. The orchestrator's whole-file invalidation reason for this case is `file-missing`, one of the canonical drift tokens listed in §19.2 Stage 1 row 2 and the Pipeline-Sidecar table: `file-missing`, `probe-failed`, `version-drift`, `alg-stamp-drift`, `epoch-drift`, `format-drift`, `recording-id-mismatch`, `config-hash-drift`, `payload-corrupt`. (The set has grown over time as new cache-key fields land — `recording-id-mismatch` was added when `.pann` files copied between recordings under the same filename had to be rejected.)
- Atomic writes via `FileIOUtils.SafeWriteBytes` — same pattern as `TrajectorySidecarBinary.Write` (`Source/Parsek/TrajectorySidecarBinary.cs:163`).
- Storage class per Section 24:
  - `SmoothingSplineList`, `OutlierFlagsList`, `AnchorCandidatesList` — annotation (deterministic from raw + algorithm-stamp version).
  - `CoBubbleOffsetTraces` — annotation when computed at commit, derived cache when computed lazily (Section 22.3 open question).

17.3.2 `.prec` Schema Changes (Phases 7 and 9 only)

Two pipeline phases require *raw* per-point fields and therefore need a `.prec` schema bump via `TrajectorySidecarBinary`. They are the only phases that touch the existing binary file.

Phase	New raw field	Placement	Format-version bump
7	`recordedGroundClearance : float32` (NaN sentinel for legacy points)	Per-point in `WritePointList` and per-section frames	`TerrainGroundClearanceBinaryVersion = 9`
9	`flags : byte`	Per-point bit-packed (default 0; bit 0 = `StructuralEventSnapshot`)	`StructuralEventFlagBinaryVersion = 10`

These bumps follow the exact pattern used today for v3, v5, v6, v7 (see `TrajectorySidecarBinary.cs:38-43`). Each adds a private const, extends `IsSupportedBinaryVersion` (`:341`), and gates the new field's read/write on a version comparison. The existing `.prec` file shape is otherwise untouched — Stages 1-5 and Phase 8 (outlier rejection) require no `.prec` changes.

The shared format constants at `RecordingStore.cs:57-61` are the canonical values — `TrajectorySidecarBinary.cs:38-43` declares its own `*BinaryVersion` constants that reference back to them, so a single bump on the `RecordingStore` side propagates automatically. Trajectory data is binary-only after the `refactor-4-pass2` series; the `.prec.txt` mirror produced by `RecordingStore.cs` is a debug-only readable dump and does not need parallel field gating.

17.3.3 What Sidecar Files Exist Per Recording

After all phases ship:

File	Existing / new	Purpose
`<id>.prec`	existing	Canonical trajectory + sections + events (binary; `TrajectorySidecarBinary.cs`). Currently at v8 (`BoundarySeamFlagFormatVersion`); schema bumped to v9 in Phase 7, v10 in Phase 9 (see 17.3.2).
`<id>.prec.txt`	existing	Optional debug-only readable mirror of `.prec`. Not consumed at load time.
`<id>_vessel.craft`	existing	Vessel proto for spawn. Untouched.
`<id>_vessel.craft.txt`	existing	Optional debug-only readable mirror.
`<id>_ghost.craft`	existing	Ghost mesh snapshot. Untouched.
`<id>_ghost.craft.txt`	existing	Optional debug-only readable mirror.
`<id>.pann`	**new**	Pipeline annotations (this design). Optional, regenerable.

(Note: `<id>.pcrf`, mentioned in earlier drafts of `.claude/CLAUDE.md`, was retired in the `refactor-4-pass2` series and now lives in `RecordingStore.LegacyRecordingFileSuffixes` for cleanup. New code does not write it.)

Older recordings without `.pann` and with `.prec` ≤ v8 remain fully loadable. The pipeline lazy-computes whatever annotations are missing (Section 21.3); per-point raw fields added in v9 / v10 read as `NaN` / `0` for older recordings, which the pipeline interprets as "data not captured" and falls back accordingly.

All new readers/writers route through `FileIOUtils.SafeWrite*` (atomic tmp + rename, HR-12) and through `RecordingPaths.Build*RelativePath` helpers (path-traversal validation).

17.4 What the Pipeline Does Not Touch

Explicit non-list, in case future work blurs the boundary:

- `ParsekScenario` `OnSave` / `OnLoad` save-file shape (`crewReplacements`, `Ledger`, `Tree*` keys, etc.). Pipeline state is transient.
- `RecordingTree` topology, `MergeState`, `SupersedeTargetId`. These are the recording-merge layer, not the rendering layer.
- `Ledger`, `RecalculationEngine`, `LedgerTombstone`. Pipeline is read-only against `Recording.CommittedRecordings`.
- `ReFlySessionMarker` durable fields. Pipeline reads them; never writes.
- `MergeJournal` and crash-recovery checkpoints. Pipeline never participates in journaled commits.
- KSP `Vessel` / `Part` / `Krakensbane` state. Pipeline reads positions only at frozen anchor UTs (HR-15).
- The grep gates (`scripts/grep-audit-ers-els.ps1`, `LogContractTests`). Pipeline's reads of `RecordingStore.CommittedRecordings` for trajectory data are not ERS/ELS-relevant; if any new code inadvertently classifies as ERS/ELS-touching, add `[ERS-exempt]` per the existing rule.

It produces:

- A `U_render(t)` per ghost per frame, consumed by the existing ghost mesh placement subsystem via `IGhostPositioner`.

It does not produce or consume:

- Persisted save-file data (no `.sfs` shape changes).
- Game state (no resource, science, contract, kerbal effects).
- Recording on-disk data outside the additive sidecar nodes listed in 17.3.

The recorder change required for Section 12 (sample-time alignment at structural events) is the only modification outside the rendering pipeline. It is additive — recordings predating it remain playable at slightly degraded anchor fidelity (Section 15.17). The sidecar additions above are also additive per the existing format-evolution rule. Older sidecars without these fields trigger lazy on-load computation gated by the new format-version constant.

---

18. Implementation Phasing

Suggested phasing for incremental rollout. Each phase is independently shippable, produces a user-visible improvement, and ends in a green `dotnet test` plus an in-game smoke test before the next phase starts. Files / types / hooks listed against each phase reference the implementation map (Section 17). All phases obey the worktree workflow (`.claude/CLAUDE.md` → "Worktree Workflow") — one feature branch per phase off `origin/main`, merge back via `git merge --no-ff`, leave the branch around unless asked to prune.

The first four phases together address the three naive-rendering failure modes (Section 3) for the most common gameplay shapes; phases 5-9 are refinements and edge-case coverage.

Phase 1: Smoothing Splines (Stage 1 only)

Goal: eliminate the jitter failure mode (3.1) on `ABSOLUTE`-frame `EXO_*` segments. No anchor work, no frame change.

New files:

- `Source/Parsek/Rendering/SmoothingSpline.cs` — `internal struct SmoothingSpline` holding `KnotsUT`, per-axis control values, type tag (Catmull-Rom v1). Methods: `internal static SmoothingSpline Fit(IList<TrajectoryPoint> samples, SegmentEnvironment env)`, `internal Vector3d EvaluatePosition(double ut)` (returns body-fixed lat/lon/alt-equivalent vector or world delta — exact contract finalized in code review).
- `Source/Parsek/Rendering/SectionAnnotationStore.cs` — `internal static` accessors over a per-recording dictionary keyed by section index. Holds `SmoothingSpline?`, `OutlierFlags?`, `AnchorCandidate[]?`, all nullable.
- `Source/Parsek/PannotationsSidecarBinary.cs` — new binary annotation sidecar (Section 17.3.1). Mirrors the shape of `Source/Parsek/TrajectorySidecarBinary.cs`: magic `PANN`, version int (`PannotationsBinaryVersion = 1` initial), `AlgorithmStampVersion` int, sidecar epoch matching the source `.prec`'s epoch, recording ID, string table. Phase 1 implements the `SmoothingSplineList` block; later phases append more blocks. Read / write through `FileIOUtils.SafeWriteBytes` (atomic tmp + rename).
- `Source/Parsek/RecordingPaths.cs` extension (additive helper, not a new file but listed here for visibility) — add `BuildAnnotationsRelativePath(string recordingId)` returning `"Parsek/Recordings/<id>.pann"`. Reuses the existing `ValidateRecordingId` guard.
- `Source/Parsek.Tests/Rendering/SmoothingSplineTests.cs` — round-trip / monotone-along-track / endpoint-value-preserved.

- `Source/Parsek.Tests/Rendering/PannotationsSidecarRoundTripTests.cs` — write / read / version-mismatch-discard / atomic-tmp-rename behaviour.

Modified files:

- `Source/Parsek/RecordingSidecarStore.cs` — wire the new annotation sidecar into the recording load / commit paths (this is the orchestrator added in `refactor-4-pass2`). On load: insert the `.pann` probe + lazy-compute call after the `.prec` deserialization succeeds (around `:235`, immediately after `RecordingStore.DeserializeTrajectorySidecar`). On commit: stage a parallel `.pann` write at the existing `SidecarFileCommitBatch.StageWrite` site (`:729`) so it lands alongside the `.prec` write under the same `rec.SidecarEpoch`. *No `.prec` schema bump in Phase 1* — the canonical recording file is untouched.
- `Source/Parsek/TrajectoryMath.cs` — add `internal static class CatmullRomFit` with `Fit(IList<TrajectoryPoint> samples, double tension) : SmoothingSpline`, `Evaluate(SmoothingSpline spline, double ut) : Vector3d`. Reuse `SanitizeQuaternion` (`:704`) shape for the coefficient defensiveness pattern.
- `Source/Parsek/ParsekFlight.cs` — at the existing `InterpolateAndPosition` body-fixed branch, swap the linear `BracketPointAtUT` (`TrajectoryMath.cs:674`) call for `SmoothingSpline.Evaluate` when an annotation is present; fall through to the existing path on miss. Single-flag rollout: a new `ParsekSettings.useSmoothingSplines` (default `true`) controls the swap.

Done condition: new xUnit tests pass; `dotnet test --filter InjectAllRecordings` produces ghost playback with no visible per-frame bouncing on a recorded coast trajectory; KSP.log carries one `[Pipeline-Smoothing]` summary per loaded recording (Section 19).

Phase 2: Live Separation Anchor (Stage 3 minimal — re-fly only)

Goal: address failure mode 3.2 for the staging re-fly case. Single live-anchor, constant ε across the affected segment. No DAG propagation yet.

New files:

- `Source/Parsek/Rendering/AnchorCorrection.cs` — `internal struct AnchorCorrection { string RecordingId; int SectionIndex; double UT; Vector3d Epsilon; AnchorSource Source; }` with `enum AnchorSource { LiveSeparation, DockOrMerge, RelativeBoundary, OrbitalCheckpoint, SoiTransition, BubbleEntry, BubbleExit, CoBubblePeer, SurfaceContinuous, Loop }`.
- `Source/Parsek/Rendering/RenderSessionState.cs` — `internal static class` holding the in-memory anchor map. Methods: `RebuildFromMarker(ReFlySessionMarker marker)`, `Lookup(string recordingId, int sectionIndex, double ut) : AnchorCorrection?`, `Clear()`. Lifetime tied to scene transitions and marker writes — populated by `RewindInvoker.ConsumePostLoad` and `ParsekScenario.OnLoad`, cleared on scene exit and re-fly clear.
- `Source/Parsek.Tests/Rendering/RenderSessionStateTests.cs` — anchor lookup before/after marker write, cleared on marker clear, deterministic across rebuild.

Modified files:

- `Source/Parsek/ParsekScenario.cs` — call `RenderSessionState.RebuildFromMarker(ActiveReFlySessionMarker)` after the existing marker-load path in `OnLoad`, and `Clear()` in `OnDestroy` / scene-transition exit.
- `Source/Parsek/RewindInvoker.cs` — `ConsumePostLoad` already runs `AtomicMarkerWrite`; immediately after, call `RenderSessionState.RebuildFromMarker`.

- `Source/Parsek/ParsekFlight.cs` — inside `InterpolateAndPosition`, after the spline evaluation from Phase 1, look up the anchor correction for the current section and add it. Wrap reads behind a feature flag `ParsekSettings.useAnchorCorrection`.

Done condition: in a re-fly scenario from a multi-controllable split, the ghost sibling spawns within centimetres of the recorded geometric attachment point at separation UT (replaces today's ~1-10 m offset). `[Pipeline-Anchor]` log emits one line per anchor evaluated.

Phase 3: Multi-Anchor Lerp (Stage 4)

Goal: address failure mode 3.3 across long segments by lerping ε between two anchors.

Modified files:

- `Source/Parsek/Rendering/AnchorCorrection.cs` — add `internal struct AnchorCorrectionInterval { AnchorCorrection Start; AnchorCorrection? End; }` and the linear-lerp evaluator.
- `Source/Parsek/Rendering/RenderSessionState.cs` — extend the anchor map to a per-section interval. The existing `Lookup` returns the interval; the renderer evaluates lerp at the current UT.
- `Source/Parsek/ParsekFlight.cs` — at the call site from Phase 2, replace the constant ε with `interval.EvaluateAt(ut)`.
- `Source/Parsek.Tests/Rendering/AnchorLerpTests.cs` — single-anchor still constant; two-anchor segment lerps linearly; lerp respects hard discontinuities (Section 11).

Done condition: an end-anchored long burn with a downstream dock no longer shows the "snap into alignment" artefact at the dock event.

Phase 4: Inertial Frame Transformation (Stage 2)

Goal: reduce along-track drift on `EXO_PROPULSIVE` and `EXO_BALLISTIC` segments.

New files:

- `Source/Parsek.Tests/Rendering/InertialLiftTests.cs` — body-rotation roundtrip (`Lift(ut)` then `Lower(ut)` is identity); Lift/Lower distributivity over add; numerical tolerance against KSP's body rotation matrix at multiple UTs.

Modified files:

- `Source/Parsek/TrajectoryMath.cs` — add `internal static Vector3d LiftToInertial(Vector3d bodyFixedWorldPos, CelestialBody body, double ut)` and the inverse `LowerFromInertial`. Implementation uses `body.bodyTransform.rotation` evaluated at `ut`, in the same shape as the rotation contract documented in `.claude/CLAUDE.md` ("Rotation / world frame"). Pure function; no side effects.
- `Source/Parsek/Rendering/SmoothingSpline.cs` — when fitting an `EXO_*` section, pre-lift samples to inertial coordinates, fit there, and tag the spline `frame = inertial`. Evaluation re-lowers at the current playback UT.
- `Source/Parsek/ParsekFlight.cs` — `InterpolateAndPosition` consults the spline tag and applies `LowerFromInertial` after `EvaluatePosition` if needed.

Done condition: an orbital coast played back across many minutes shows no progressive along-track drift relative to the recorded `OrbitSegment` propagation. Inertial-mode fits do not regress body-fixed playback for `Atmospheric` / `Surface*` sections (Phase 1 path unchanged for those).

Phase 5: Co-Bubble Overlap Blend (Stage 5)

Goal: sub-metre fidelity at very close range. Both live-primary and ghost-primary cases.

New files:

- `Source/Parsek/Rendering/CoBubbleBlender.cs` — designated-primary selector (Section 10.1) and `EvaluateOffset(string recordingId, double ut, out Vector3d offset)`. Uses cached `CoBubbleOffsetTraces` entries from `<id>.pann` (Section 17.3.1).
- `Source/Parsek/Rendering/CoBubbleOverlapDetector.cs` — commit-time pass that scans pairs of recordings whose lifespans overlap and produces `CoBubbleOffsetTraces` entries when they share a physics bubble. Lazy fallback: same routine triggered on first co-render.
- `Source/Parsek.Tests/Rendering/CoBubbleBlenderTests.cs` — primary selection priority, snapshot-freezing of the live primary's reference, bounded crossfade at window exit.

Modified files:

- `Source/Parsek/PannotationsSidecarBinary.cs` — extend the binary schema with `CoBubbleOffsetTraces` per Section 17.3.1 (no `.prec` change). Bump `AlgorithmStampVersion` on classifier change.
- `Source/Parsek/ParsekFlight.cs` — `InterpolateAndPosition` consults `CoBubbleBlender.TryEvaluate` first; on hit, computes `P_render(t)` via the standalone Stages-1+2+3+4 path against the primary's recording and returns `P_render(t) + offset(t)` (Section 6.5 / Section 10); on miss, falls through to the standalone spline + ε path from Phases 1-4. Live-primary case uses the same call — `P_render(t)` reads from the primary's recording, never the live vessel's runtime KSP state.

Done condition: decoupling debris stays visually attached to the parent vessel for the recorded relative geometry; formation-flying ghosts maintain centimetre-accurate spacing across orbital motion (the primary's canonical motion advances `P_render(t)` so the peer follows). Player input on the live primary does not move the peer ghost.

Phase 6: Anchor Taxonomy Completion (Stage 3 full)

Goal: every anchor type in Section 7 produces an `AnchorCorrection`. DAG propagation per Section 9 walks `BranchPoint` edges (`Source/Parsek/BranchPoint.cs:18`) without per-event special-casing.

New files:

- `Source/Parsek/Rendering/AnchorCandidateBuilder.cs` — commit-time scan of `TrackSection` s, `BranchPoint` s, `OrbitSegment` s, and the existing v7 absolute-shadow data. Emits `AnchorCandidate[]` per section.
- `Source/Parsek/Rendering/AnchorPropagator.cs` — DAG walk over `RecordingTree` edges starting from each live anchor. Produces ε for downstream segments per Section 9.1.
- `Source/Parsek.Tests/Rendering/AnchorPropagationTests.cs` — three-stage rocket re-fly (Section 9.2); cross-recording dock at chain tip (Section 9.3); suppressed predecessor (Section 9.4).

Modified files:

- `Source/Parsek/PannotationsSidecarBinary.cs` — extend the schema with the `AnchorCandidatesList` block per Section 17.3.1. Bump `AlgorithmStampVersion` for any classifier change so older `.pann` files are discarded and recomputed (HR-10). No `.prec` change.
- `Source/Parsek/Rendering/RenderSessionState.cs` — invoke `AnchorPropagator.Run(tree, marker)` during `RebuildFromMarker` .

Done condition: every anchor type from Section 7.1-7.10 is exercised by an in-game test (`InGameTests/RuntimeTests.cs` , see Section 20). Suppressed-subtree filtering verified.

Phase 7: Continuous Terrain Correction (Section 13)

Goal: `SurfaceMobile` ghosts neither float nor clip when terrain mesh shifts between sessions.

This phase bumps the `.prec` binary schema because it captures a genuinely new raw field (`recordedGroundClearance`) at every sample. The bump follows the existing pattern in `Source/Parsek/TrajectorySidecarBinary.cs:38-43` .

Modified files:

- `Source/Parsek/TrajectoryPoint.cs` — additive field `public double recordedGroundClearance` (default `double.NaN` for legacy points; readers fill NaN when the field is absent in older binaries).
- `Source/Parsek/RecordingStore.cs` — append a new format constant `TerrainGroundClearanceFormatVersion = 9` to the version block (currently ends at `BoundarySeamFlagFormatVersion = 8`); bump `CurrentRecordingFormatVersion` to it.
- `Source/Parsek/TrajectorySidecarBinary.cs` — append `TerrainGroundClearanceBinaryVersion = RecordingStore.TerrainGroundClearanceFormatVersion` to the version constants block. Extend `IsSupportedBinaryVersion` and `GetBinaryEncoding` . Gate the new field's read/write inside `WritePointList` / `ReadPointList` on `binaryVersion >= TerrainGroundClearanceBinaryVersion` . Trajectory data is binary-only (post `refactor-4-pass2`); the `.prec.txt` debug mirror is regenerated from the binary representation and inherits the new field automatically — no parallel text-codec changes required.
- `Source/Parsek/FlightRecorder.cs` — populate `recordedGroundClearance` for every sample in a `SurfaceMobile` section. Use `body.pqsController.GetSurfaceHeight(...)` minus the recorded altitude at recording time (or the equivalent KSP-API distance to surface).
- `Source/Parsek/ParsekFlight.cs` — `PositionAtPoint` and `InterpolateAndPosition` for `SurfaceMobile` sections use `body.pqsController.GetSurfaceHeight` plus `recordedGroundClearance` instead of stored `altitude` when `recordedGroundClearance` is non-NaN; otherwise fall through to today's altitude path.
- `Source/Parsek/Rendering/TerrainCacheBuckets.cs` — lat/lon-bucketed cache, evicted at scene transition.

API substitution note: the implementation calls `body.TerrainAltitude(lat, lon, true)` rather than `body.pqsController.GetSurfaceHeight(QuaternionD.AngleAxis(...) * radial)` directly. `TerrainAltitude` is the higher-level wrapper that calls `pqsController.GetSurfaceHeight` plus ocean clamping (the `withOcean=true` flag) and matches the prior art in `VesselSpawner.ClampAltitudeForLanded` . Both forms produce identical metres-above-mean-radius output for SurfaceMobile rover terrain; the wrapper is preferred for ocean-bearing bodies and code-path consistency. Future implementers should not "fix" this back to the lower-level call.

Done condition: a recorded rover replayed in a second session over the same terrain stays at constant ground clearance; no clipping at procedural-mesh seams. Older recordings (format ≤ 7) load with `recordedGroundClearance = NaN` and use today's altitude path — backwards-compatible.

Phase 8: Outlier Rejection (Section 14)

Goal: kraken-event resilience.

New files:

- `Source/Parsek/Rendering/OutlierClassifier.cs` — environment-aware acceleration / bubble-radius / altitude-range thresholds.
- `Source/Parsek/Rendering/OutlierFlags.cs` — packed-bitset annotation type.

Modified files:

- `Source/Parsek/PannotationsSidecarBinary.cs` — extend the schema with the `OutlierFlagsList` block per Section 17.3.1; bump `AlgorithmStampVersion` on classifier change so older `.pann` files are discarded and recomputed (HR-10). No `.prec` change — outlier flags are derived data.
- `Source/Parsek/Rendering/SmoothingSpline.cs` — `Fit` consumes the flag bitset and skips rejected points.

Done condition: synthetic kraken-injected recording (`InjectAllRecordings`) plays back smoothly through the spike.

Phase 9: Sample-Time Alignment at Structural Events (Section 12, recorder-side)

Goal: every `BranchPoint` UT carries a synchronized snapshot for every involved vessel. New recordings only — old recordings keep their existing fidelity (Section 15.17).

This phase bumps the `.prec` binary schema because it captures a genuinely new raw field (the `flags` byte tagging structural-event snapshots). Same pattern as Phase 7.

Modified files:

- `Source/Parsek/TrajectoryPoint.cs` — additive `flags` byte (default 0 for legacy points; bit 0 = `StructuralEventSnapshot`). A new `[Flags] enum TrajectoryPointFlags : byte` documents the bit assignments.
- `Source/Parsek/RecordingStore.cs` — append `StructuralEventFlagFormatVersion = 10` to the version block; bump `CurrentRecordingFormatVersion` to it.
- `Source/Parsek/TrajectorySidecarBinary.cs` — append `StructuralEventFlagBinaryVersion = RecordingStore.StructuralEventFlagFormatVersion` to the version constants. Extend `IsSupportedBinaryVersion` and `GetBinaryEncoding` . Gate the new `flags` byte read/write inside `WritePointList` / `ReadPointList` on `binaryVersion >= StructuralEventFlagBinaryVersion` . No parallel text-codec change (trajectory is binary-only post-refactor; `.prec.txt` is the debug mirror only).
- `Source/Parsek/FlightRecorder.cs` — at the dock / undock / EVA / `onPartJointBreak` (`PartJoint joint, float breakForce`) handlers, call a new `AppendStructuralEventSnapshot(double eventUT, IEnumerable<Vessel> involved)` that interpolates each vessel's per-tick state to the exact event UT and writes one `TrajectoryPoint` per involved vessel into the corresponding section, with `flags |= TrajectoryPointFlags.StructuralEventSnapshot` . Both vessels' snapshots are taken from the same physics state (Section 12).
- `Source/Parsek/Rendering/AnchorCandidateBuilder.cs` — prefer `StructuralEventSnapshot` -flagged points over interpolated samples when computing event ε. Recordings without the flag fall through to today's interpolation behaviour.
- `Source/Parsek.Tests/Generators/RecordingBuilder.cs` — extend with `WithStructuralEventSnapshot(double ut, ...)` so test fixtures can produce v10 recordings.

Done condition: post-Phase-9 recordings show physics-precision ε at every structural event in `BranchPoint` -driven anchors. Older recordings (format ≤ 8) load without the `flags` byte and the pipeline falls back to interpolated event ε (Section 15.17).

Phase Ordering Discipline

- Phases 1-4 are sequential and high-leverage. Each builds on the previous and unlocks a visible failure-mode fix.
- Phases 5-9 are independent and can ship in any order once 1-4 land. None block each other.
- Format-version landings: Phase 1 introduces the `.pann` annotation sidecar at its own initial version `PannotationsBinaryVersion = 1`; later phases that touch `.pann` bump only the `AlgorithmStampVersion` inside the file, never `PannotationsBinaryVersion`. The canonical `.prec` chain currently ends at `BoundarySeamFlagFormatVersion = 8` on `main`; Phases 7 and 9 reserve v9 (`TerrainGroundClearanceFormatVersion`) and v10 (`StructuralEventFlagFormatVersion`) respectively (Section 17.3.2). Phases 2-6 and 8 do not touch `.prec`.
- Each phase ends with a CHANGELOG entry under the current Parsek version (per `.claude/CLAUDE.md` → "Documentation Updates — Per Commit, Not Per PR") and the corresponding `docs/dev/todo-and-known-bugs.md` strikethroughs.

---

19. Diagnostic Logging

Per `.claude/CLAUDE.md` → "Logging Requirements", every action, state transition, guard skip, and FX lifecycle event must be logged. The pipeline is no exception. This section enumerates the log lines each stage and each anchor type emits, the subsystem tag to use, and the level.

All log calls go through the existing `ParsekLog` API (`Source/Parsek/ParsekLog.cs`):

```
ParsekLog.Info(string subsystem, string message);                                    //
state transition / one-shot
ParsekLog.Warn(string subsystem, string message);                                    //
recoverable degradation
ParsekLog.Verbose(string subsystem, string message);                                 //
gated by ParsekSettings.verboseLogging
ParsekLog.VerboseRateLimited(string subsystem, string key, string message,
                             double minIntervalSeconds = 5.0);                       //
per-frame summaries
ParsekLog.VerboseOnChange(string subsystem, string identity, string stateKey,
                          string message);                                           //
state-change-only
```

Format is fixed by `ParsekLog.Write`: `[Parsek][LEVEL][Subsystem] message`.

19.1 Subsystem Tags

The pipeline uses one tag per stage so a developer can grep a single concern in isolation. All tags are prefixed `Pipeline-` to keep them distinct from existing subsystems (`Engine`, `Recorder`, `Rewind`, `Marker`, `Spawner`, `MapPresence`, etc.).

Tag	Owns
`Pipeline-Smoothing`	Stage 1 spline fits, evaluations, fallback to legacy bracket
`Pipeline-Frame`	Stage 2 inertial lift / lower decisions
`Pipeline-Anchor`	Stage 3 anchor identification, type, source, ε magnitude
`Pipeline-AnchorPropagate`	Section 9 DAG walk

`Pipeline-Lerp`	Stage 4 interval evaluation, both-end lerp vs constant
`Pipeline-CoBubble`	Stage 5 primary selection, blend window, crossfade
`Pipeline-Outlier`	Section 14 rejection counts, classifier hits
`Pipeline-Terrain`	Section 13 raycast cache hits / misses
`Pipeline-Session`	`RenderSessionState` rebuild / clear / invalidation
`Pipeline-Sidecar`	New annotation node read / write / lazy compute
`Pipeline-Format`	Format-version gating, degraded-mode entry

19.2 Per-Stage Log Lines

For each stage, the table below lists the events that must produce a log line, the level, the tag, and the data the line must include. A reviewer should be able to reconstruct the stage's behaviour from the log alone — no source code reading required (per `.claude/CLAUDE.md` 's "if it didn't get logged, it didn't happen").

Stage 1 — Smoothing

Event	Level	Tag	Data to include
Spline fit at commit	Info	`Pipeline-Smoothing`	`recordingId`, sectionIndex, env, sampleCount, knotCount, fit duration ms
Spline fit lazy (annotation absent / drift on load)	Info	`Pipeline-Smoothing`	`recordingId`, sectionIndex, reason ∈ {`file-missing`, `version-drift`, `epoch-drift`, `format-drift`, `config-hash-drift`, `alg-stamp-drift`, `recording-id-mismatch`}
Fallback to legacy bracket (fit failure)	Warn	`Pipeline-Smoothing`	`recordingId`, sectionIndex, reason, sampleCount
Spline evaluation per frame (rate-limited)	Verbose	`Pipeline-Smoothing`	`VerboseRateLimited` shared key, count summary
Settings flag flip (`useSmoothingSplines`)	Info	`Pipeline-Smoothing`	old → new

Stage 2 — Frame Transformation

Event	Level	Tag	Data to include
Section lift to inertial decision	Verbose	`Pipeline-Frame`	`recordingId`, sectionIndex, env, framing chosen
Lift / lower mismatch (round-trip > tol)	Warn	`Pipeline-Frame`	`recordingId`, sectionIndex, ut, residual metres

Stage 3 — Anchor Identification

Event	Level	Tag	Data to include

Anchor candidate computed at commit	Verbose	`Pipeline-Anchor`	`recordingId`, sectionIndex, candidateUT, candidateType
Anchor ε computed at session entry	Info	`Pipeline-Anchor`	`recordingId`, sectionIndex, side (start/end), source, ε magnitude m
Anchor source priority resolution	Verbose	`Pipeline-Anchor`	UT, candidates considered, winner per Section 7.11
Anchor missing → constant ε = 0	Verbose	`Pipeline-Anchor`	`recordingId`, sectionIndex, reason
Live anchor read at frozen UT	Verbose	`Pipeline-Anchor`	`recordingId`, anchorUT, frozenWorldPos
Anchor ε exceeds bubble radius (sanity)	Warn	`Pipeline-Anchor`	`recordingId`, sectionIndex, side, magnitude, anchorSource (HR-9)

Stage 3b — Anchor Propagation (Section 9)

Event	Level	Tag	Data to include
DAG walk start	Info	`Pipeline-AnchorPropagate`	rootRecordingId, treeId, marker.SessionId
Edge propagated	Verbose	`Pipeline-AnchorPropagate`	parent → child recordingId, branchPointType, ε delta
Suppressed predecessor skipped	Verbose	`Pipeline-AnchorPropagate`	recordingId, reason="suppressed"
DAG walk summary	Info	`Pipeline-AnchorPropagate`	edges visited, anchors set, terminal reasons (no-successor / suppressed)

Stage 4 — Correction Interpolation

Event	Level	Tag	Data to include
Interval lerp evaluated (rate-limited)	Verbose	`Pipeline-Lerp`	shared `VerboseRateLimited` key; counts and avg ε per ghost
Single-anchor → constant ε	Verbose	`Pipeline-Lerp`	`recordingId`, sectionIndex, side held
ε divergence ε_start vs ε_end (Section 8)	Warn	`Pipeline-Lerp`	`recordingId`, sectionIndex, divergence m, segment length s

Stage 5 — Co-Bubble Blend

Event	Level	Tag	Data to include
Primary selection	Info	`Pipeline-CoBubble`	recordingA, recordingB, primary, rule (1-4 from Section 10.1)

Blend window enter	Info	`Pipeline-CoBubble`	primary, peer, startUT, expectedEndUT
Blend window exit	Info	`Pipeline-CoBubble`	primary, peer, exitUT, reason (separation / time / destruction / TIME_JUMP)
Crossfade frame (rate-limited)	Verbose	`Pipeline-CoBubble`	shared `VerboseRateLimited` key, blend ratio
Trace miss → standalone fallback	Verbose	`Pipeline-CoBubble`	recordingId, ut, reason

Outlier Rejection (Section 14)

Event	Level	Tag	Data to include
Sample rejected	Verbose	`Pipeline-Outlier`	`recordingId`, sampleIndex, classifier, value vs threshold
Per-section rejection summary	Info	`Pipeline-Outlier`	`recordingId`, sectionIndex, rejectedCount / total, classifier breakdown
Cluster threshold exceeded → low-fidelity tag	Warn	`Pipeline-Outlier`	`recordingId`, sectionIndex, rejectionRate

Terrain (Section 13)

Event	Level	Tag	Data to include
Cache miss + raycast	Verbose	`Pipeline-Terrain`	lat/lon bucket, body, hit / miss
Raycast miss → fallback to recorded altitude	Warn	`Pipeline-Terrain`	`recordingId`, lat, lon, body
Cache eviction at scene transition	Info	`Pipeline-Terrain`	bucket count cleared

Session State Lifecycle

Event	Level	Tag	Data to include
`RebuildFromMarker` start	Info	`Pipeline-Session`	marker.SessionId, marker.ActiveReFlyRecordingId
`RebuildFromMarker` complete	Info	`Pipeline-Session`	recordings indexed, anchors, ε map size, duration ms
`Clear`	Info	`Pipeline-Session`	reason (scene-exit / marker-clear / re-fly-end)
Stale-state detection during render	Warn	`Pipeline-Session`	what was stale, what triggered the rebuild request

Sidecar I/O

Event	Level	Tag	Data to include
New node read OK	Verbose	`Pipeline-Sidecar`	`recordingId`, nodeName, version, byteCount
Node missing → lazy compute scheduled	Info	`Pipeline-Sidecar`	`recordingId`, nodeName, reason
Whole-file invalidation → discard + recompute	Info	`Pipeline-Sidecar`	`recordingId`, reason ∈ {`version-drift`, `alg-stamp-drift`, `epoch-drift`, `format-drift`, `recording-id-mismatch`, `config-hash-drift`, `payload-corrupt`, `file-missing`, `probe-failed`}, found vs expected
Per-trace co-bubble invalidation	Info	`Pipeline-CoBubble`	`recordingId`, peerRecordingId, reason (peer-missing / peer-epoch-changed / peer-format-changed / peer-content-mismatch)
Atomic write success	Verbose	`Pipeline-Sidecar`	`recordingId`, nodeName, byteCount
Atomic write failure (HR-12)	Warn	`Pipeline-Sidecar`	`recordingId`, nodeName, IO error

Format Gating

Event	Level	Tag	Data to include
Recording loaded with pre-pipeline format	Info	`Pipeline-Format`	`recordingId`, formatVersion, degraded-mode list
Annotation algorithm version drift	Info	`Pipeline-Format`	`recordingId`, found version, current version, action taken

19.3 Batch Counter Convention

Per the project convention (`.claude/CLAUDE.md` → "Batch counting convention"), per-frame iterations log a single summary, not per-item. Pipeline counters (added in `GhostPlaybackEngine` next to the existing batch counters):

```
// Per-frame summary, reset each frame
int frameSplineEvalCount;
int frameAnchorLookupCount;
int frameAnchorMissCount;
int frameLerpEvalCount;
int frameCoBubbleEvalCount;
int frameTerrainRaycastCount;
int frameTerrainCacheHitCount;
```

Logged via `VerboseRateLimited` with shared key `"pipeline-frame-summary"` at 1.0 s intervals: `splineEvals=N anchorLookups=M anchorMisses=X lerpEvals=L coBubbleEvals=C terrainRays=T (cacheHits=H)` .

19.4 Failure Visibility (HR-9)

The pipeline never silently substitutes a wrong-but-plausible result. Every fallback path emits at least one `Warn` line at `Pipeline-<Stage>` tag. Specific cases:

- Spline fit fails → `Pipeline-Smoothing` Warn, fall through to existing `BracketPointAtUT`.
- Anchor `ε` magnitude exceeds bubble radius → `Pipeline-Anchor` Warn, ε kept (not zeroed) but flagged as suspect.
- Co-bubble trace empty mid-window → `Pipeline-CoBubble` Verbose then standalone fallback.
- Terrain raycast misses on `SurfaceMobile` → `Pipeline-Terrain` Warn, recorded altitude used.
- Outlier-cluster rate exceeds threshold → `Pipeline-Outlier` Warn, low-fidelity tag set on diagnostics screen.

The `Warn` lines are also covered by `LogContractTests` (`Source/Parsek/InGameTests/LogContractTests.cs`): each new tag added in this section gets a contract entry asserting the format is `[Parsek][WARN][Pipeline-<Stage>] ...`.

---

20. Test Plan

Every test in this plan has a "what makes it fail" justification. Tests without one would not catch a real bug — per `.claude/CLAUDE.md` → "Testing Requirements" they are forbidden as vacuous.

The Parsek test infrastructure offers three test layers; the pipeline uses all three:

1. **xUnit unit tests** (`Source/Parsek.Tests/`) — pure logic, data transformations, serialization round-trips. xUnit working dir is `Source/Parsek.Tests/bin/Debug/net472/`; classes touching shared static state need `[Collection("Sequential")]` and the corresponding `ResetForTesting()` calls (per `.claude/CLAUDE.md`).
2. **xUnit log-assertion tests** — capture log output via `ParsekLog.TestSinkForTesting` and assert specific lines exist. Verifies behaviour AND that diagnostic coverage survives refactoring.
3. **In-game tests** (`Source/Parsek/InGameTests/`) — `[InGameTest(Category = "...", Scene = GameScenes.FLIGHT)]`. Run via Ctrl+Shift+T. For things that require live KSP (PartLoader, terrain mesh, krakensbane).

Test fixtures use the existing generators (`Source/Parsek.Tests/Generators/`):

- `RecordingBuilder` — fluent recording fixtures. Extend with `WithTrackSection(env, refFrame, ...)` if not already present, `WithBranchPoint(BranchPointType, ut, ...)`, `WithSplineCoefficients(...)`, `WithOutlierFlags(...)`, `WithCoBubbleOffsets(peerRecordingId, ...)`. Additive — existing tests continue to pass.
- `VesselSnapshotBuilder` — vessel ConfigNode fixtures. The PID convention `100000 + idx*1111` is mandatory for ghost part lookup (`.claude/CLAUDE.md` → "Ghost event ↔ snapshot PID"). Single-part showcase ghosts must use PID `100000`.
- `ScenarioWriter` — synthetic save fixtures including a `RenderSessionState`-friendly marker.

20.1 Unit Tests (xUnit, Pure Logic)

Test	What makes it fail	File / location

`CatmullRomFit` round-trip on uniform samples	Linear ramps not exactly preserved → indicates fit destroys real dynamics	`Source/Parsek.Tests/Rendering/SmoothingSplin`
`CatmullRomFit` endpoint preservation	Spline at section start / end does not pass through the boundary sample → would misalign anchor	same
`CatmullRomFit` no smoothing across hard discontinuities	Spline spans a `BranchPoint` UT → violates HR-7 (Section 11)	same
`LiftToInertial` / `LowerFromInertial` round-trip	`Lower(Lift(p)) - p` exceeds tolerance → frame transform numerically unstable	`InertialLiftTests.cs`
`LiftToInertial` distributivity over add	Inertial lift not linear → would bias lerp	same
`AnchorCorrection` constant single-side	One-sided anchor produces non-constant ε → would visibly drift	`AnchorLerpTests.cs`
`AnchorCorrection` two-sided lerp	Lerp output not linear in `ut` → would distribute drift unevenly	same
`AnchorCorrection` priority order	Higher-priority source not preferred at same UT → Section 7.11 violated	`AnchorPriorityTests.cs`
`AnchorPropagator` three-stage rocket (Section 9.2)	L2+U start ε differs from L1 live + recorded offset → propagation rule violated	`AnchorPropagationTests.cs`
`AnchorPropagator` cross-recording dock	Downstream recording's start ε ignores upstream's end ε at dock UT → Section 9.3 broken	same
`AnchorPropagator` suppressed predecessor	Suppressed segment's ε leaks into non-suppressed successor → HR-8 violated	same

`CoBubbleBlender` primary selection	Tied ghosts produce non-deterministic primary → HR-3 violated; would flicker across save/load	`CoBubbleBlenderTests.cs`
`CoBubbleBlender` snapshot freezing	Live primary's reference moved with player input → naive-relative trap (Section 3.4)	same
`CoBubbleBlender` exit-condition crossfade	No fade at window exit → visible snap	same
`OutlierClassifier` rejects implausible accel	Atmospheric-rocket-ceiling acceleration accepted → kraken sample fits into spline	`OutlierTests.cs`
`OutlierClassifier` accepts plausible accel	High-burn acceleration falsely rejected → recording loses real dynamics	same
`.pann` round-trip `SmoothingSplineList`	Save / load mutates coefficients → silent drift	`PannotationsSidecarRoundTripTests.cs`
`.pann` round-trip `OutlierFlagsList`	Bit-packing inconsistent → wrong samples rejected on second load	same
`.pann` round-trip `CoBubbleOffsetTraces`	Offset arrays diverge → ghost-ghost geometry drifts after load	same
`.pann` round-trip `AnchorCandidatesList`	UT array misordered after save / load → anchor lookups skip valid candidates	same
`.pann` `AlgorithmStampVersion` mismatch discards file	Stale annotations from old algorithm leak into new playback → silent wrong rendering	`PannotationsVersionGatingTests.cs`
`.pann` `SourceSidecarEpoch` mismatch discards file	`.pann` from a previous `.prec` epoch is consumed → cache-key drift bug	same
`.pann` `ConfigurationHash` reproducibility	Same tunables produce different hashes on different runs → cache	`PannotationsConfigHashTests.cs`

	becomes useless or HR-10 violated silently	
`.pann` `ConfigurationHash` sensitivity (perturb each tunable)	Tunable mutated without changing hash → stale cache accepted as fresh, HR-10 broken	same
Co-bubble per-trace peer epoch validation	Peer `.prec` rewritten (heal / supersede / re-record); existing trace must be discarded, not silently accepted	`CoBubblePeerCacheKeyTests.cs`
Co-bubble per-trace peer format-version validation	Peer `.prec` migrated to a newer format; existing trace must be discarded	same
Co-bubble per-trace `peerContentSignature` mismatch	Peer raw bytes in `[startUT, endUT]` window changed under same epoch (defence-in-depth) → trace discarded	same
Co-bubble peer-missing fallback	Peer recording deleted; trace must discard and consumer falls back to standalone, with HR-9 Warn log	same
`.prec` v9 / v10 schema round-trip	New per-point fields (`recordedGroundClearance`, `flags`) corrupt on save / load of newer recordings	`PrecBinaryRoundTripTests.cs`
`.prec` v8 read under v9 code	Older recording missing `recordedGroundClearance` reads as NaN; pipeline must fall back gracefully	same
`.prec` v9 read under pre-v9 code	Older Parsek build encounters newer `.prec` and refuses cleanly (`probe.Supported = false`) — no silent corruption	same
`RenderSessionState.RebuildFromMarker` deterministic	Same marker + same recordings → different ε map between runs → HR-3 violated	`RenderSessionStateTests.cs`

`RenderSessionState.Clear` invalidation	Anchor lookup after `Clear` returns stale value → would cause ghost to spawn at last-session position	same

20.2 Log-Assertion Tests (xUnit, `TestSinkForTesting`)

These verify both *behaviour* and *diagnostic coverage*. If the test passes today and a future refactor accidentally drops the log line, the assertion fails and the developer must restore the log. Pattern (per `RewindLoggingTests.cs`):

```
public sealed class PipelineSmoothingLoggingTests : IDisposable
{
    private readonly List<string> logLines = new();
    public PipelineSmoothingLoggingTests() {
        ParsekLog.TestSinkForTesting = line => logLines.Add(line);
    }
    public void Dispose() => ParsekLog.ResetTestOverrides();
    // ... [Fact] methods asserting Assert.Contains(logLines, l => l.Contains("[Pipeline-
Smoothing]") && l.Contains("..."))
}
```

Test	What makes it fail
Spline fit at commit logs once per section	Refactor breaks `Verbose` per-section summary → silent loss of commit-time diagnostics
Anchor identification logs ε magnitude	A future change drops the "ε=...m" formatting → reviewer cannot judge anchor health from logs
Anchor priority resolution log	Multiple-source UT no longer logs the chosen winner → Section 7.11 becomes invisible
Anchor ε > bubble-radius emits Warn	Sanity guard silently degrades to `Verbose` → bug masquerades as normal operation
Lerp divergence emits Warn	Long-segment frame mismatch loses its visibility
`RenderSessionState.RebuildFromMarker` start + end	Either bookend missing → impossible to time session-state rebuild from logs
Sidecar version-mismatch logs `Info` and recomputes	Silent recompute → cache-key drift bug loses its only audit trail
Co-bubble window enter / exit pair	Exit log dropped → blend window leaks across recordings without trace
Outlier-cluster Warn at high rejection rate	Threshold tuning loses telemetry; recordings of unknown fidelity ship without flag
Frame summary `VerboseRateLimited` shared key	New per-item Verbose log added in pipeline hot path causes log spam → caught by absence of summary

20.3 In-Game Tests (`InGameTests/RuntimeTests.cs`)

Categories added (each appears in `parsek-test-results.txt`):

- `Pipeline-Smoothing` — load a synthetic recording with known jitter; assert per-frame position delta < threshold.
- `Pipeline-Anchor-LiveSeparation` — synthetic re-fly scenario; assert ghost spawn position within `ε_live ≤ 0.05 m` of recorded geometric attachment.
- `Pipeline-Anchor-RelativeBoundary` — RELATIVE / ABSOLUTE seam; assert no visible snap at the boundary.
- `Pipeline-Anchor-OrbitalCheckpoint` — burn-end / coast-start; assert ε bracketed by checkpoint-derived position.
- `Pipeline-Anchor-SOI` — synthetic SOI-crossing recording; assert correct body for each side.
- `Pipeline-Anchor-BubbleEntry` — recording with mid-record bubble exit / re-entry.
- `Pipeline-CoBubble-Live` — re-fly with co-bubble peer; assert centimetre-accurate spacing.
- `Pipeline-CoBubble-GhostGhost` — both ghosts present, no live; assert formation geometry preserved.
- `Pipeline-Terrain-Continuous` — `SurfaceMobile` ghost across a procedural-mesh seam; assert no clipping over a 30-second window.
- `Pipeline-Outlier-Kraken` — synthetic single-frame teleport sample; assert spline does not deflect.
- `Pipeline-Loop-Anchor` — anchored loop near a live vessel; assert no drift across cycles.
- `Pipeline-DAG-Three-Stage` — Section 9.2 scenario.
- `Pipeline-Determinism` — same recording loaded twice; assert byte-identical world positions across a 5-second sample.

Coroutine-style tests (multi-frame `IEnumerator`) are appropriate for any test that observes drift or stability over time.

20.4 Synthetic Recordings

The pipeline's edge cases are best exercised by synthetic recordings injected into a test save via `dotnet test --filter InjectAllRecordings` (per `.claude/CLAUDE.md`). The pipeline ships eight new fixtures, parallel to existing synthetic recordings:

1. `pipeline-smoothing-coast.prec` — 60 s coast at LKO with synthetic jitter envelope.
2. `pipeline-anchor-separation.prec` — staging split with one ghost sibling, geometric offset known to the millimetre.
3. `pipeline-anchor-dock.prec` — dock event mid-recording.
4. `pipeline-soi-crossing.prec` — Kerbin → Mun SOI mid-burn.
5. `pipeline-cobubble-formation.prec` — two co-bubble vessels in tight formation.
6. `pipeline-terrain-rover.prec` — rover circumnavigating a 100 m radius near KSC.
7. `pipeline-outlier-kraken.prec` — single-frame 10 km teleport sample.
8. `pipeline-loop-rover.prec` — looped rover with persistent anchor at KSC pad.

Each fixture lives under `Source/Parsek.Tests/Fixtures/Pipeline/` and is built via `RecordingBuilder` so the fixture is re-derivable; the on-disk `.prec` is regenerated by a `dotnet test --filter "RegenerateFixtures"` target.

20.5 Per-Phase Test-Done Checklist

Maps to Section 18's phase ordering. A phase is not done until every box ticks.

Phase	xUnit unit tests	Log-assertion tests	In-game tests	Sidecar round-trip	Synthetic fixtures
1	Spline fit + endpoint + no-cross-discontinuity	Smoothing fit + fallback + summary	`Pipeline-Smoothing`	`.pann SmoothingSplineList`	`pipeline-smoothing-coast`
2	`RenderSessionState` lifecycle, single anchor	Anchor identification + ε magnitude	`Pipeline-Anchor-LiveSeparation`	(none new)	`pipeline-anchor-separation`
3	Lerp linearity, divergence Warn	Lerp divergence Warn	`Pipeline-Anchor-RelativeBoundary`	(none new)	(reuse)
4	Lift / lower round-trip, distributivity	Frame mismatch Warn	`Pipeline-Anchor-OrbitalCheckpoint`	(none new)	(reuse)
5	Primary selection, P_render frame coupling, crossfade	Co-bubble enter / exit pair	`Pipeline-CoBubble-Live` + `-GhostGhost`	`.pann CoBubbleOffsetTraces`	`pipeline-cobubble-formation`
6	Three-stage propagation, suppressed predecessor	DAG walk summary	`Pipeline-DAG-Three-Stage`	`.pann AnchorCandidatesList`	(reuse)
7	Terrain bucket cache hit / miss, eviction	Terrain Warn on raycast miss	`Pipeline-Terrain-Continuous`	`.prec` v9 round-trip + v8→v9 read	`pipeline-terrain-rover`
8	Classifier accept / reject, cluster threshold	Outlier rejection summary, cluster Warn	`Pipeline-Outlier-Kraken`	`.pann OutlierFlagsList`	`pipeline-outlier-kraken`
9	Structural-event snapshot tagged correctly	Recorder snapshot Info line	(recorder-side, in-game test verifies precision)	`.prec` v10 round-trip + v9→v10 read (`TrajectoryPoint.flags` byte)	(regenerate fixtures with v10 flag)

20.6 Test Anti-Patterns the Pipeline Avoids

- `Assert.NotNull(spline)` without checking values → vacuous.
- `Assert.True(logLines.Count > 0)` without checking the line content → vacuous.
- Tests that only exercise the happy path while Section 15 lists 17 edge-case scenarios → inadequate.
- Tests that mock the recording sidecar I/O → masks atomic-write bugs (HR-12). Always round-trip via real `FileIOUtils.SafeWrite` and `RecordingStore`.

- Tests that hard-code today's algorithm constants (smoothing tension, outlier thresholds) → break on Section 22.1's empirical re-tuning. Test the contract (monotonic, bounded, deterministic), not the constant.

---

21. Backward Compatibility & Format Evolution

The pipeline's only persistent additions are sidecar annotation nodes (Section 17.3). All recordings produced before the pipeline's introduction remain loadable and playable; the pipeline degrades gracefully across format generations.

21.1 Format Version Chain

Two version chains are relevant to the pipeline. Keep them distinct in any review or implementation discussion.

`.prec` (canonical recording, `Source/Parsek/TrajectorySidecarBinary.cs`). Magic `PRKB`. Trajectory data is binary-only post-`refactor-4-pass2`; `.prec.txt` is a debug-only readable mirror, not consumed at load. The shared format version constants live at `RecordingStore.cs:57-61`; `TrajectorySidecarBinary.cs:38-43` declares matching `*BinaryVersion` constants that reference back, so a single bump on the `RecordingStore` side propagates. Each existing version is also a binary version; the pipeline appends two more — but only in Phases 7 and 9.

Constant	Value	What it gates
`LaunchToLaunchLoopIntervalFormatVersion`	4	Loop interval = launch-to-launch period (not post-cycle gap)
`PredictedOrbitSegmentFormatVersion`	5	`OrbitSegment.isPredicted` is serialized
`RelativeLocalFrameFormatVersion`	6	RELATIVE-section frames carry anchor-local offsets in `TrajectoryPoint` lat/lon/alt
`RelativeAbsoluteShadowFormatVersion`	7	RELATIVE-section adds `absoluteFrames` shadow
`BoundarySeamFlagFormatVersion`	8	`TrackSection.isBoundarySeam` flag for the Producer-C no-payload boundary seam (already shipped on `main`)
`TerrainGroundClearanceFormatVersion` *(new, Phase 7)*	9	Per-point `recordedGroundClearance : double` for `SurfaceMobile` samples
`StructuralEventFlagFormatVersion` *(new, Phase 9)*	10	Per-point `flags : byte` (bit 0 = `StructuralEventSnapshot`)

`CurrentRecordingFormatVersion` advances to 8 in Phase 7 and 9 in Phase 9. Phases 1-6 and 8 do *not* bump it. Any new behaviour gated on a version comparison uses the named constants — raw integer comparisons rot when the next version lands (per `.claude/CLAUDE.md` → "Format-v7 enums").

`.pann` (pipeline annotations, new file `Source/Parsek/PannotationsSidecarBinary.cs`). Magic `PANN`. Independent version chain.

Constant	Value	What it gates
`PannotationsBinaryVersion` *(new, Phase 1)*	1	Initial `.pann` schema (Section 17.3.1) — `SmoothingSplineList`, `OutlierFlagsList`, `AnchorCandidatesList`, `CoBubbleOffsetTraces` blocks. Phases 1, 6, 8, 5 each populate one block; the file's binary version itself does not bump until a structural schema change.

`AlgorithmStampVersion` *(file-internal int)*	1+	Bumps every time a smoothing / outlier / anchor algorithm changes its output for the same input. Older `.pann` files with a non-matching stamp are discarded and recomputed (HR-10).

The two chains are independent. A recording can be at `.prec` v8 with no `.pann` (lazy compute), or at `.prec` v10 with `.pann` v1 (everything eager), or any combination. Cache-key freshness is verified by matching the `.pann` `SourceSidecarEpoch` and `SourceRecordingFormatVersion` fields against the source `.prec` 's values.

21.2 What Each Older Recording Produces Through the Pipeline

Loaded `.prec` format	Pipeline behaviour
≤ 5	RELATIVE sections route through the legacy v5-and-older code path (per `.claude/CLAUDE.md`). Smoothing + anchors lazy-computed and persisted to a fresh `.pann` on first load.
6	RELATIVE anchor-local offsets resolve via the existing `TrajectoryMath.ApplyRelativeLocalOffset`. No `absoluteFrames` shadow → live-anchor RELATIVE sections cannot use the v7 bypass; pipeline still runs Stages 1-5 on ABSOLUTE sections. `.pann` lazy-computed.
7	All current behaviours. Stages 1-5 enabled on ABSOLUTE sections; v7 absolute shadow continues to feed `ResolveAbsoluteShadowPlaybackFrames` for Re-Fly RELATIVE bypass. `.pann` lazy-computed unless already on disk.
8 (Phase 7+)	Plus per-point `recordedGroundClearance`. `SurfaceMobile` ghosts use continuous terrain correction (Section 13). Older `SurfaceMobile` sections (no clearance) fall back to today's altitude-only path.
9 (Phase 9+)	Plus per-point `flags` byte. Structural-event snapshots used by `AnchorCandidateBuilder` for physics-precision event ε. Older recordings (no flag) use interpolated event ε (Section 15.17).

21.3 Lazy-Compute Path

For any recording without a current-stamp `.pann` on disk, the pipeline computes annotations on first read. The whole-file invalidation cases that trigger lazy compute are:

- File absent.
- `PannotationsBinaryVersion` not in the supported set (binary schema bumped).
- `AlgorithmStampVersion` mismatch (deterministic algorithm changed output for same input).
- `SourceSidecarEpoch` mismatch against the source `.prec` (source recording was rewritten).
- `SourceRecordingFormatVersion` mismatch (source `.prec` migrated across format versions).
- `ConfigurationHash` mismatch (any tunable in the canonical encoding changed — Section 17.3.1's "Configuration Cache Key" enumerates them).

The flow:

1. `RecordingStore.LoadRecording` reads `.prec` as today.
2. After deserialization, `PannotationsSidecarBinary.TryProbe(<id>.pann)` runs. On any whole-file mismatch above (returning `Success = false` with `FailureReason` set), the pipeline executes `FitSmoothingSplinesPerSection` , `OutlierClassifier.Run` , `AnchorCandidateBuilder.Run` .
3. `CoBubbleOverlapDetector.Run` is partial: traces that pass the per-trace peer-cache-key check (Section 17.3.1's "Per-Trace Peer Validation") are kept; failing traces are recomputed individually. A `.pann` whose

splines / outliers / anchors are still fresh but whose co-bubble traces are stale is rewritten with the recomputed traces only, preserving the still-valid blocks.
4. Results land in an in-memory `SectionAnnotationStore` keyed by `recordingId`.
5. By default the pipeline persists the freshly-computed annotations back to `<id>.pann` via `PannotationsSidecarBinary.Write` (atomic tmp + rename, HR-12). The canonical `.prec` is *not* modified — only the new sibling `.pann` is written. Recordings remain immutable on disk per HR-1.
6. Setting `ParsekSettings.skipLazyPipelinePersist = true` keeps `.pann` as in-memory only (debug / test scenario). Default off; running with the default persists the cache so subsequent loads are O(1) probe + read.

`Pipeline-Format` Info logs document each degraded-mode entry; `Pipeline-Sidecar` logs document each lazy compute and each `.pann` write (Section 19.2). Per-trace co-bubble invalidations log under `Pipeline-CoBubble` with the specific reason (peer epoch / peer format / peer content signature / peer absent).

21.4 Cross-Version Re-Fly

A re-fly session can reference recordings of mixed formats (a tree built up across mod versions). The pipeline:

- Treats every recording's section through its version-appropriate frame contract.
- Builds anchors per Section 7 wherever the data exists. Older recordings missing data simply contribute fewer anchors.
- Logs `Pipeline-Format` Info per recording listing which pipeline features are active for it.
- Renders correctly in all combinations; older recordings just look like the pre-pipeline behaviour.

This mirrors the existing rule for v5 vs v6 RELATIVE handling — version dispatch happens per-section, never globally.

21.5 Save-File Compatibility

The pipeline writes nothing to `.sfs` save files. `ParsekScenario.OnSave` / `OnLoad` are unchanged. Sharing a save with a player on a pre-pipeline build:

- The pre-pipeline build never opens `<id>.pann` (it has no code path for it). The file is harmless on disk — it sits unread.
- For `.prec` v9 / v10 recordings sent to a pre-Phase-7 / pre-Phase-9 build, the older `TrajectorySidecarBinary` reader fails the version check (`IsSupportedBinaryVersion` returns false). The recipient sees `probe.FailureReason = "unsupported binary trajectory version 9"` (or v10) and the recording fails to load gracefully — no silent corruption, no plausible-but-wrong data. Players should upgrade the receiving side before consuming v9 / v10 recordings.

There is no migration tool. Recordings flow through versions by additive evolution; downgrades are a non-goal (consistent with the existing project policy).

21.6 What This Section Is Not

This section does not specify a runtime "compatibility mode" toggle visible to the player. There is no toggle — the pipeline is on for every recording it can use, and the diagnostic logs make degraded-mode entries observable. Section 22 (open questions) covers cases where the right behaviour is genuinely uncertain; that is not a compatibility concern.

---

22. Open Questions

These are areas where the design is intentionally underspecified pending implementation experience.

22.1 Smoothing Window Heuristics

The exact spline type (Catmull-Rom vs Hermite vs B-spline), tension parameters, and outlier threshold tuning are deferred. They can be tuned empirically once Increment 1 is in.

22.2 Designated-Primary Stability

When two co-bubble ghosts are roughly equal in priority (Section 10.1 tiebreakers), the primary designation should be stable across save/load. Whether this requires persistence or just deterministic tie-breaking from recording metadata is open.

22.3 Trace Precomputation Strategy

Co-bubble offset traces (Section 10.2) can be precomputed at commit time (cost: storage) or computed lazily on first co-bubble overlap (cost: latency on re-fly start). Choice depends on typical recording size and overlap frequency. Likely lazy with persistent cache after first computation.

22.4 Surface Terrain Cache Granularity

Section 13.2's lat/lon cache bucket size — driven by raycast cost vs cache memory. Empirical tuning.

22.5 Long Time Warps Across Mixed-Frame Segments

Time warp through a session containing both ABSOLUTE and ORBITAL_CHECKPOINT segments interleaved by short bursts of physics — does the rendering keep up across all warp rates, or do certain ghosts stutter? Depends on KSP1's warp-tick architecture. Empirical.

22.6 Cross-Session Visual Consistency

Section 15.16 (shared saves) — what level of cross-machine visual consistency is achievable, and is it worth additional engineering to make it exact? Currently best-effort.

22.7 Anchor Conflicts

Section 7.11 establishes priority for multiple-source anchors. Whether any real-world recording produces a true conflict (e.g., a live vessel and an analytical orbit both sourcing the same UT and disagreeing meaningfully) is unclear. The priority order resolves it but the magnitude of typical disagreement is unknown until data exists.

---

23. Vocabulary

Term	Meaning
Anchor	A UT at which a high-fidelity reference position for a ghost is available.
Common-mode noise	Sampling errors shared between two co-bubble vessels; cancels in their difference.
Designated reference / primary	The vessel against which another is rendered relative to during co-bubble overlap. Can be live or ghost.
ε (epsilon) / correction	Rigid-translation vector added to a ghost's smoothed position to align it with an anchor.

Frame transformation	Render-time switch between body-fixed and body-centered inertial representations.
Hard discontinuity	A point along a trajectory across which smoothing splines and correction lerps must not cross.
Lerp	Linear interpolation of correction values between two anchors bracketing a segment.
Pipeline	The five-stage process producing rendered ghost positions from recorded data.
Smoothing	Catmull-Rom or Hermite spline fit through recorded samples per segment.
Snapshot	A reference vessel's position frozen at an anchor UT, used for co-bubble blend.
Stored offset trace	Recorded `non_primary_abs(t) - primary_abs(t)` over a co-bubble overlap window.

---

24. Data Lifecycle and Storage

24.1 Data Classes

The pipeline operates on data of four distinct provenance classes. Confusing them is the single most common source of correctness bugs in pipelines of this shape.

Class	Source	Mutability	Persistence
Raw recorded	Captured by recorder during gameplay	Immutable after commit	Persistent (sidecar)
Annotation	Computed from raw at commit time	Append-only across versions	Persistent (sidecar)
Derived cache	Pure function of raw + annotation + configuration	Recomputable on demand	Persistent or transient (implementation choice)
Session state	Computed from raw + annotation + live game state	Transient	In-memory only

Every byte the pipeline reads or writes belongs to exactly one class. A reviewer should be able to identify the class of any data by its location and access pattern. If a value is hard to classify, that is a design smell — usually it means raw data and derived data have been conflated.

24.2 Raw Recorded Data (Immutable)

What the recorder captures and what is saved to disk as canonical truth:

- **Trajectory samples**: position and velocity at recorded UTs, in their original recording frame.
- **Sample timing**: the exact UT each sample was taken.
- **Structural event snapshots**: synchronized samples at split, merge, dock, undock, joint-break UTs (Section 12).
- **SegmentEvents and BranchPoints**: with their exact UTs, types, and metadata.
- **Environment and reference-frame designations**: per track section, as captured at recording time.
- **Ground clearance**: per surface sample, captured at recording time.
- **Terminal state**: how the recording ended.

- **Vessel identity metadata**: controller list, persistent IDs.
- **Antenna specs**: for CommNet.
- **Recording start/end UTs**.

After commit, none of this changes. There is no migration, no normalization, no "fixing" of recorded data. If the recorder wrote a sample with a known imperfection, the imperfection stays in the raw record. The pipeline corrects for it at render time via annotation and session-state layers.

The recorder writes raw data to memory during a session, then flushes it to disk at commit. The flush is the only moment raw data crosses from volatile to persistent. Once persisted, it is read-only.

24.3 Annotation Data (Append-Only, Versioned)

Annotations are facts derived from raw data that augment it without contradicting it. They are computed at commit time (or lazily at first read) and stored alongside raw data in the sidecar.

Examples:

- **Outlier-rejection markers**: which raw samples are physically implausible (Section 14). The samples themselves remain in raw data; the annotation is a parallel array of flags.
- **Smoothing spline coefficients**: per-segment fit through non-rejected samples. The spline is an annotation; the samples it was fitted from are unchanged.
- **Co-bubble overlap windows**: which UT ranges had which other vessels in physics bubble. Computed by scanning recordings.
- **Anchor candidate index**: per-segment list of UTs that are anchor opportunities. Pre-computed for fast lookup.
- **Pre-lifted inertial-frame samples**: lifts of raw samples into the segment's render frame, for hot-path performance. The raw body-fixed values remain alongside.

Annotations are append-only across format versions. New annotation types may be added; old ones never have their semantics changed silently. If an annotation type's algorithm changes, it is a new annotation type with a new version, and old data either remains valid or is recomputed on demand.

If annotations are not present (e.g., the recording predates this pipeline), they are computed lazily on first read and cached. The recording remains valid; the pipeline degrades to compute-on-demand.

24.4 Derived Cache (Recomputable)

Caches are pure functions of raw + annotation + configuration constants (smoothing tension, outlier thresholds, terrain bucket size, etc.). They exist for performance.

Properties of a valid cache entry:

- Recomputable from inputs at any time, deterministically.
- Bit-identical to a fresh recompute, given the same inputs and configuration.
- Safe to delete at any time without losing information.

Examples that may be cached:

- Co-bubble offset traces (Section 10.2): pure function of two raw sample series.
- Outlier rejection results under a fixed threshold configuration.
- Pre-lifted inertial-frame sample arrays.
- Anchor candidate UT lists.

Caches may be persisted to the sidecar for performance, but the sidecar format must distinguish caches from raw and annotation data. A reader encountering a cache it cannot validate (version mismatch, configuration mismatch)

discards it and recomputes.

24.5 Session State (Transient)

Anything that depends on live game state is session state. It lives in memory for the duration of a session and is never persisted.

Examples:

- Anchor `ε` values: depend on live vessel positions.
- Designated-primary assignments: depend on which ghosts are loaded.
- Frozen reference snapshots: depend on when re-fly started.
- Per-frame rendered ghost positions.

Session state is rebuilt on every session entry from raw + annotation + cache + current game state. If a save/load cycle happens, session state is reconstructed from scratch.

24.6 The Memory-to-Disk Boundary

During an active recording session, raw samples are accumulated in memory at recorder rate. No annotation, cache, or session state exists for the in-progress recording — the recording is incomplete.

At commit, raw samples are finalized and written to disk; annotations are computed and written alongside; caches may be eagerly written or deferred to first-read. The recording transitions from in-progress to immutable.

Between sessions, raw and annotation data persist on disk, caches may persist on disk, and session state is gone.

On load, raw and annotation data are read; caches are validated against current algorithm version and configuration and used if valid (otherwise recomputed); session state is rebuilt from scratch based on the loaded game state.

24.7 Format Evolution

Sidecar files carry a version header. The format evolves additively per the existing additive-format-evolution rule from the flight recorder doc.

For pipeline-related additions:

- New raw fields (e.g., velocity samples if not previously recorded): additive. Old recordings remain valid; the pipeline computes velocity from position deltas where samples are missing, marked as derived.
- New annotation types: additive. Computed lazily for older recordings.
- New cache types: additive. Computed on demand if absent.

Sidecar version bumps for pipeline additions follow the existing scheme.

---

25. Post-Commit Optimization Opportunities

This section is forward-looking. The base pipeline (Sections 1-23) is correct without any of these. They are opportunities to be considered after the base pipeline is shipping and stable.

25.1 The Live-Anchor Coupling

Most of the pipeline's complexity comes from the possibility of a live vessel existing alongside ghosts:

- Anchor `ε` values depend on the live vessel's spawn position.
- Designated-primary assignments may include the live vessel.
- Co-bubble blend windows reference frozen snapshots taken from live state.

- The propagation chain through the DAG often starts from a live anchor.

When no live vessel is in the relevant ghost network — pure-ghost playback after a recording is committed and no re-fly is active — this coupling vanishes. The remaining pipeline operates entirely on raw + annotation data.

This is the regime where the most aggressive optimizations are safe.

25.2 Static Anchor Closure

Without a live anchor, every anchor in a connected ghost network is one of:

- An analytical-orbit anchor (orbital checkpoint, SOI transition).
- A persistent-vessel anchor (RELATIVE-frame docking, looped recording).
- A DAG-propagated anchor from one of the above.

These are pure functions of raw + annotation data. Their `ε` values are fixed and can be computed once when the ghost set is determined, then reused across all frames until the ghost set changes.

The "ghost set changes" trigger is rare in pure-ghost playback — typically only at zone transitions (a ghost enters or exits the bubble) or at chain-tip spawn (a ghost materializes as real). Between such triggers, the entire `ε` map is constant. A re-fly start invalidates it.

25.3 Pre-Computed Render Tracks

Taking 22.2 further: without a live anchor, the rendered position of a ghost at any UT is a deterministic function of (recording, UT). The full render track for a recording can be precomputed and cached as a closed-form expression in UT.

For ABSOLUTE-frame segments, this collapses to spline + constant `ε` (or lerp `ε`) + frame transform. For ORBITAL_CHECKPOINT segments, it is already analytical. For RELATIVE-frame segments, it depends on the anchor vessel's current position (a single game-state read per frame).

A pre-computed render track is a function `render_position(UT)` evaluated in constant time. Re-fly start invalidates any track whose segments are touched by the new live anchor.

25.4 Aggregation Across Recordings

Multiple committed recordings forming a single ghost network (linked by dock/merge events) share a common anchor closure (22.2). The aggregated network has a single set of anchors and a single set of pre-computed render tracks.

When the player loads a save, the entire visible ghost network can be aggregated into one rendering data structure with shared cache. More efficient than per-recording caches when many recordings interact.

25.5 Per-Distance-Zone Sample Reduction

A committed recording's raw samples are stored at the rate they were captured. For ghost rendering at distance, this is over-sampled — a ghost at 50 km does not need physics-tick-rate position data.

A safe optimization: per-distance-zone reduced sample arrays. Compute reduced-resolution arrays at commit time (Zone 2 lower rate, Zone 3 lower still) by uniform sub-sampling. Use the reduced arrays for distant rendering; full-resolution only when in physics bubble.

This is annotation, not modification of raw data. The full-resolution samples remain available for any computation that needs them, including a hypothetical zoom-in or a re-load into the bubble.

25.6 Cross-Recording Smoothing Independence (Non-Optimization)

Smoothing splines are computed per-segment per-recording. There is no opportunity to smooth across recording boundaries even when their geometry is geometrically continuous (e.g., a docking event linking two recordings).

The recorded relative offsets at boundaries are exact (Section 4); the splines on each side terminate at the boundary value. Smoothing across the boundary would introduce error — the boundary is exactly where the two recordings agree.

This is called out as a non-optimization to forestall the temptation. Do not be tempted.

25.7 Cache Sharing Across Sessions

If a player saves and loads, derived caches (Section 24.4) remain valid as long as the algorithm version and configuration match. Session-entry cost reduces to: read raw + annotation + cache, validate, rebuild only session state.

Cache keys must include (algorithm version, configuration hash). A cache with a non-matching key is discarded silently and recomputed.

25.8 Risks of Aggressive Optimization

Each optimization removes a recompute. If the recompute is wrong (algorithm bug, configuration mismatch), the cached result perpetuates the wrong answer indefinitely. Mitigations:

- Every cache must be recomputable at any time. Discarding caches and recomputing must be cheap and routine, not a recovery operation.
- A "cache verification" mode (debug/test only) recomputes from scratch and compares against the cache, flagging mismatches.
- Cache keys hash not just version but also a content-derived signature of inputs that detects accidental skew.

The base pipeline ships without these optimizations. Each is added incrementally with verification scaffolding around it.

---

26. Hard Rules and Risk Surface

This section enumerates correctness invariants that must hold across the full pipeline (recorder → processor → save → load → post-processor → renderer). Violations are bugs by definition; many are easy to introduce and hard to debug.

26.1 Hard Rules

The pipeline is bound by the following non-negotiable rules. Every line of pipeline code must be auditable against this list.

HR-1. Raw samples are immutable after capture. The recorder writes a sample once. After it lands in raw data, it is never modified, normalized, transformed, or interpolated in place. Any "improvement" to a sample produces an annotation or a derived cache, never a change to the raw value.

HR-2. No lies about provenance. Any value the pipeline produces that did not come directly from a recorded sample must be tagged as derived. Smoothed positions are not samples. Interpolated positions at non-sample UTs are not samples. Anchor-corrected positions are not samples. Every consumer must be able to distinguish "what was recorded" from "what we compute for display."

HR-3. Determinism. Given the same raw data, annotation data, configuration, and (where applicable) live game state, the pipeline produces the same output bit-for-bit. No randomness, no thread-ordering dependencies, no time-

of-day dependencies, no uninitialized memory.

HR-4. Idempotence. Running any pipeline stage twice on the same input produces the same output. Computing annotations on already-annotated data produces the same annotations. Building session state on top of session state already built produces the same state.

HR-5. Read-only against game state and recordings. The pipeline does not mutate game state, vessel state, save files, or recording data. Its only writes are to its own annotation, cache, and session-state stores.

HR-6. No retroactive modification of past renders. Once a frame has been rendered, the rendering of subsequent frames does not depend on the previous frame's render output. Each frame is computed from current inputs alone. Stored carryover is limited to configuration constants and cached annotations, neither of which is "previous frame state."

HR-7. No smoothing or anchoring across hard discontinuities. Sections 11 and 6.1 are absolute. A spline does not span a structural event, environment transition, frame transition, SOI crossing, bubble entry/exit, TIME_JUMP, or recording boundary. An anchor lerp does not span any of the same.

HR-8. No suppressed-data influence. Suppressed segments (rewind-to-staging suppression) do not contribute to any computation that affects non-suppressed output: not anchors, not splines, not designated-primary selection, not aggregation.

HR-9. Failure is visible, not silent. If the pipeline cannot produce a correct rendering (missing data, version mismatch, algorithm error), it falls back to a clearly-labeled degraded mode and logs the failure. It does not produce a plausible-looking-but-wrong result.

HR-10. Configuration changes invalidate caches. Any tunable parameter (smoothing tension, outlier threshold, terrain bucket size, etc.) is part of the cache key. Changing a parameter without invalidating affected caches is a correctness bug.

HR-11. Annotations are append-only across versions. New annotation types may be added; old annotation semantics are never silently changed. If an algorithm changes, it is a new annotation type or a version bump.

HR-12. The recorder commits or aborts atomically. A partially-committed recording is a corrupted recording. Either all of (raw samples + annotations + format header) lands on disk, or none does. The reader must be able to detect partial commits and reject them.

HR-13. No DAG cycles. The DAG is acyclic by construction; the pipeline assumes it. Any code path that walks the DAG must terminate even if a cycle is somehow introduced (programmer error). Cycle protection in the walk is defensive.

HR-14. No cross-recording sample contamination. Smoothing splines, outlier rejection, and per-segment computations operate on a single recording's data. They do not borrow samples from other recordings even when those recordings are co-bubble. Cross-recording information enters only as anchor offsets (Sections 9, 10), stored as separate annotation, never as substituted samples.

HR-15. Live state is read at anchor UT only and frozen. Any time the renderer reads live vessel state for use as a reference, it reads it once at the appropriate anchor UT and freezes the value. Reading live state every frame and using it as the reference re-introduces the naive-relative trap (Section 3.4).

26.2 Risk Surface by Pipeline Stage

The pipeline has seven stages. Each has characteristic failure modes.

Stage A — Recorder

- **Mutating samples after capture.** Forbidden by HR-1.
- **Missing structural-event snapshots.** Section 12 requires synchronized samples at every structural event. Missing them silently degrades anchor fidelity at exactly the moment the pipeline most needs it.
- **Sample-time inconsistency.** If different vessels' samples claim the same UT but were taken at different real ticks, common-mode cancellation breaks. Snapshots must use a single physics state.
- **Buffering loss.** Samples held in memory between physics tick and flush may be lost on crash. The commit-or-abort rule (HR-12) bounds the damage to in-progress recordings.

Stage B — Processor (commit-time)

- **Writing derived data into raw fields.** A processor that "normalizes" sample positions and stores normalized values in raw arrays violates HR-1.
- **Outlier deletion.** Removing rejected samples from the raw array (instead of marking them in annotation) loses data permanently.
- **Non-deterministic ordering.** Processing samples in an order that depends on thread scheduling produces different annotations on different runs. Violates HR-3.
- **Configuration drift.** If outlier thresholds or smoothing parameters are not pinned to the annotation's version, recomputing later with different parameters produces inconsistent results.

Stage C — Optimizer (post-commit consolidation)

- **Lossy aggregation.** Merging multiple recordings' data into a shared structure may lose per-recording provenance. The optimizer must preserve traceability from any aggregated value to its source recording.
- **Sample reduction without preservation.** Section 25.5's distance-zone sample reduction must keep the full-resolution data accessible. Discarding it is a permanent loss.
- **Premature optimization.** Optimizations that bet on what the renderer will need (e.g., "this segment will never be in physics bubble again") are bets, not facts. They can be wrong.

Stage D — Save to Disk

- **Partial writes.** A crash mid-write leaves a corrupted sidecar. Mitigation: write to a temporary file, then atomic rename (HR-12).
- **Format-version drift.** Writing a new format version's data with an old version's header. The header must be the last thing written, or a checksum must verify body matches header.
- **Cache poisoning.** Writing caches without including the cache key (algorithm version + configuration). Future loads pick up stale caches and apply them to new code.

Stage E — Reader

- **Silent unknown-field tolerance.** Forward compatibility allows ignoring unknown fields, but ignoring a field that affects correctness produces wrong results. The reader must distinguish "unknown field, safe to ignore" from "unknown field, correctness-relevant" via format-version gating.
- **Cache trust without validation.** Loading a cache and using it without verifying the cache key matches the current algorithm version + configuration produces wrong results indistinguishable from correct ones. Every cache load is gated on a key check; mismatches discard the cache.
- **Partial-load tolerance.** Reading a recording where some sidecar fields are missing should produce a known degraded state, not undefined behavior. The reader fills missing fields with sentinel values that the rest of the pipeline knows to handle.

Stage F — Post-Processor (load-time)

- **Re-derivation differs from original derivation.** If the post-processor recomputes annotations using a different algorithm than the one that produced the saved annotations, the result is inconsistent. Mitigation: annotation algorithm versions are pinned; if an annotation is present and its version matches, it is trusted; otherwise it is recomputed from scratch using the current algorithm.

- **Transitive cache invalidation.** Changing a low-level parameter that affects many caches must invalidate all of them. The cache dependency graph must be explicit.

Stage G — Renderer

- **Stale session state.** Session state may become stale during a session (e.g., a new ghost enters bubble). The renderer must detect state-changing events and rebuild affected session state.
- **Frame-to-frame drift.** Accumulating any per-frame quantity (e.g., "smoothed velocity from last frame") instead of recomputing from inputs produces drift over many frames. Violates HR-6.
- **Live state coupling that escapes the snapshot rule.** Reading live state every frame as the anchor reference re-introduces the naive-relative trap. Violates HR-15.
- **Correction propagation lag.** When session state changes, the rebuild must complete before the next render. A render with partially-rebuilt state shows visible jumps. State rebuilds are atomic from the renderer's perspective — either the old state or the new state is in effect, never a half-updated mix.

26.3 Diagnostic Checklist

When debugging a pipeline failure (visual artifact, ghost in wrong place, ghost missing), walk this checklist top-down. Each step isolates one class of bug.

1. **Is the raw data correct?** Read the raw samples directly, bypassing all pipeline stages. Are positions and UTs as expected? If no, recorder bug (Stage A).
2. **Are structural events correctly recorded?** Check that every dock/undock/separation has a synchronized snapshot pair at the exact event UT for both involved vessels. If no, Stage A bug or pre-Section-12 recording.
3. **Are annotations correct?** Read the annotation data directly. Are smoothing splines fit to the right samples? Are outlier flags consistent with the samples? If no, processor bug (Stage B).
4. **Are caches matching the configuration?** Check cache keys against current algorithm version and configuration. If mismatched, cache invalidation bug (Stages D/E).
5. **Are anchors identified correctly?** For each segment, list the anchor UTs and their reference-position sources. Is each anchor type from Section 7? If wrong, anchor-identification bug (Stage F or session state).
6. **Are anchor `ε` values reasonable?** Each `ε` should be small (sub-bubble-radius). Large ones indicate either a frame mismatch (Stage G) or a propagation bug (Section 9).
7. **Is the lerp configuration correct?** For segments with two anchors, the lerp should be smooth. If it jumps, the lerp formula is wrong or the anchor UTs are mis-ordered.
8. **Is session state stale?** Check whether the artifact persists across a full session-state rebuild. If a rebuild fixes it, session-state invalidation logic is missing a trigger (Stage G).
9. **Is suppression honored?** If a ghost from a suppressed segment is visible, suppression filtering is broken (HR-8).
10. **Is determinism preserved?** Render the same scene twice with the same inputs. If results differ, HR-3 is violated somewhere.

A bug not isolated by this checklist is likely either in the rendering layer below the pipeline (mesh placement, material setup) or in interaction with KSP itself (frame timing, physics state). Those are outside the pipeline's risk surface.

26.4 Forbidden Operations (Quick List)

A one-line summary of operations that must never appear in pipeline code. Suitable as a grep checklist for code review.

- Modifying a raw sample's position, velocity, or UT after commit.
- Deleting a raw sample (outliers are flagged in annotation, not removed).
- Storing a smoothed or interpolated value in a raw-data field.
- Reading live game state every frame as an anchor reference (must be a frozen snapshot).
- Smoothing across a hard discontinuity (Section 11).
- Propagating anchors through a suppressed segment.
- Borrowing samples from another recording for a per-segment computation.
- Computing any annotation with thread-order or time-of-day dependence.
- Writing a sidecar non-atomically.
- Trusting a cache without validating its key.
- Filling a missing sample by inventing a position; either skip the frame or use a sentinel.
- Modifying ghost game-state (resources, science, contracts, kerbals) in any way.
- Smoothing across a recording boundary even when geometry is continuous.
- Discarding full-resolution samples when computing reduced-resolution arrays.

A code review surfacing any of these in pipeline code is a probable bug.

26.5 What This Section Is Not

This section does not describe a runtime enforcement mechanism. The hard rules are correctness obligations on the implementing code, not assertions to be checked at runtime (with rare exceptions, e.g., sidecar format validation on read). Most rules are not amenable to runtime checks — there is no efficient way to verify "we did not modify a raw sample" at every access; the obligation is on the structure of the code.

The diagnostic checklist (23.3) is for debugging a known artifact, not for routine validation. Routine validation is the test suite's job, scoped per stage.

---

27. Quick Reference for Coding Agents

This section is a non-normative summary intended as a code-base navigation aid. The normative content is in Sections 1-26.

To answer "where should smoothing live?": Stage 1, applied at recording-commit time, output cached in the recording sidecar. See Sections 5 and 6.1.

To answer "what frame should this segment render in?": Look up the segment's environment in the existing taxonomy and consult the Section 6.2 table. The pipeline is driven by the taxonomy, not by new metadata.

To answer "where do anchors come from?": Section 7. Every entry has a trigger and a reference-position source. New code that introduces a new structural event or boundary type should add an entry here.

To answer "how do I propagate anchors through the DAG?": Section 9. The propagation is a single tree walk along DAG edges, with one rule per edge (Section 9.1).

To answer "what about save/load?": Pipeline state is transient. Rebuild from recording data on every session entry. See Sections 16.2 and 16.3.

To answer "should I change the recording format?": No, except for Section 12's sample-time alignment (recorder-side, additive) and Section 16.2's sidecar caches (additive sidecar fields). Recordings remain forward- and backward-compatible.

To answer "where do I detect frame discontinuities?": Section 11 lists every type. Each is already represented in existing data (DAG events, taxonomy fields, checkpoints). The pipeline is a consumer of these signals, not a definer.

To answer "how do I handle a new ghost type?": If it has a continuous-reference anchor (like loops or surface mobile), it goes into Section 7 as a continuous-anchor entry and the pipeline treats it the same. If it's a new discontinuity type, it goes into Section 11.

To answer "what is allowed to mutate state?": Render-time computations are read-only against recordings and game state. The only mutations are to transient pipeline caches (Section 16.2) and to the rendered ghost mesh transforms. Hard rules are in Section 26.1.

To answer "what data is recorded vs derived vs cached?": Section 24. Four classes — raw recorded (immutable), annotation (append-only, on disk), derived cache (recomputable, optional on disk), session state (transient, in-memory only). Each byte the pipeline reads or writes belongs to exactly one class.

To answer "what can I optimize after commit?": Section 25. Biggest wins come when no live vessel exists alongside ghosts — the `ε` map becomes static and full render tracks can be precomputed as closed-form functions of UT. Per-distance-zone sample reduction (25.5) is the next-biggest. Cross-recording smoothing is explicitly forbidden (25.6).

To answer "what must I never do?": Section 26.4 has the forbidden-operations grep list. Section 26.1 has the 15 hard rules. Section 26.3 is the diagnostic checklist for debugging an artifact.

To answer "I'm adding a new pipeline stage / changing the recorder / introducing a new annotation type": Read Section 26.2 (Risk Surface by Pipeline Stage) for that stage's known failure modes before writing code. Each stage has a characteristic set of bugs.